

H I N T

HMG Incubation
of New Talent

2026년 현대자동차 K-뉴딜 아카데미

임베디드AI (SW)



실시간 운영체제 기반 전자제어기 프로그래밍

Contents

실시간 운영체제 기반 전자제어기
프로그래밍

RTOS 기본 개념
FreeRTOS 커널 배포판
힙(Heap) 메모리 관리
Task 관리

FreeRTOS 기본 개념

실시간 운영체제

□ Multitasking in Small Embedded Systems

○ FreeRTOS 커널 소개

- FreeRTOS는 실시간 커널(Real-Time Kernel)과 다양한 기능을 제공하는 모듈형 C 라이브러리로 구성된 소프트웨어이다.
 - FreeRTOS는 Richard Barry가 약 2003년에 처음 개발하였으며, 이후 Real-Time Engineers Ltd.가 주요 반도체 기업들과 협력하여 개발을 하였다.
 - 2016년부터 Amazon Web Services(AWS)가 FreeRTOS 프로젝트를 관리하고 있으며, FreeRTOS는 MIT 라이선스 기반의 오픈소스 소프트웨어로, AWS 사용 여부와 관계없이 누구나 사용할 수 있다.
 - FreeRTOS 커널은 특히 마이크로컨트롤러(MCU) 또는 소형 마이크로프로세서에서 동작하는 임베디드 실시간 시스템에 적합하며, 이러한 시스템에서는 일반적으로 Hard Real-Time과 Soft Real-Time 요구사항이 함께 존재한다.
- 참고
 - Soft Real-Time 시스템 – 정해진 시간 제한(deadline)이 존재하지만, 이를 초과하더라도 시스템 전체가 완전히 실패하는 것은 아니다. 예를 들어 키보드 입력에 대한 반응이 다소 늦으면 사용자가 시스템을 답답하게 느낄 수 있지만, 시스템 자체를 사용할 수 없는 것은 아니다.
 - Hard Real-Time 시스템 – 정해진 시간 제한을 반드시 만족해야 하며, deadline을 초과하면 시스템의 심각한 실패로 이어질 수 있다. 예를 들어 자동차의 에어백(Airbag)이 충돌 센서의 입력에 너무 늦게 반응한다면 오히려 탑승자에게 더 큰 피해를 줄 수 있다.

실시간 운영체제

□ Multitasking in Small Embedded Systems (계속)

○ FreeRTOS 역할

- FreeRTOS 커널은 애플리케이션이 Hard Real-Time 요구사항을 만족할 수 있도록 하는 실시간 스케줄러(Real-Time Scheduler)이다.
- 애플리케이션을 여러 개의 독립적인 실행 흐름(Thread of Execution)으로 구성할 수 있도록 지원하며, FreeRTOS에서는 이러한 실행 단위를 일반적으로 Task라고 부른다.
- 단일 코어 프로세서에서는 한 시점에 하나의 Task만 실행할 수 있기 때문에, FreeRTOS 커널의 Scheduler는 각 Task에 부여된 Priority(우선순위)를 기반으로 어떤 Task를 실행할 것인지 결정한다.
 - 높은 우선순위 -> Hard Real-Time Task
 - 낮은 우선순위 -> Soft Real-Time Task

실시간 운영체제

□ Why Use an RTOS?

간단한 임베디드 시스템은 RTOS 없이도 충분히 구현할 수 있지만 시스템이 복잡해질수록 RTOS Kernel을 이용하는 것이 애플리케이션 구조와 실시간 요구사항 관리 측면에서 더 유리할 수 있다.

○ 시간 제어의 추상화 (Abstracting Away Timing Information)

➤ RTOS가 Task의 실행 시점과 시간 관리를 담당한다. 애플리케이션은 RTOS가 제공하는 시간 관련 API를 이용하므로 프로그램 구조를 단순화할 수 있으며 코드 크기도 줄일 수 있다.

- Application -> RTOS API -> Scheduling/timing -> Hardware

○ 유지보수성과 확장성 (Maintainability / Extensibility)

➤ 시간과 관련된 처리를 RTOS가 담당하면 각 소프트웨어 모듈 사이의 의존성을 줄일 수 있다. 따라서 소프트웨어를 보다 체계적이고 예측 가능한 방식으로 수정하고 확장할 수 있다. 또한 하드웨어가 변경되더라도 애플리케이션의 동작 특성이 받는 영향을 줄일 수 있다.

○ 모듈화 (Modularity)

➤ 각 Task는 서로 독립적인 모듈로 설계할 수 있으며, 각각 명확한 목적을 갖도록 구성하는 것이 좋다.

- Task 1: 센서 데이터 수집
- Task 2: 데이터 처리
- Task 3: 통신 처리
- Task 4: 사용자 인터페이스

실시간 운영체제

□ Why Use an RTOS? (계속)

○ 팀 개발에 유리 (Team Development)

- 각 Task가 명확한 인터페이스를 가지도록 설계하면 여러 개발자가 서로 다른 Task를 독립적으로 개발할 수 있다.

○ 테스트가 용이 (Easier Testing)

- 독립적으로 구성된 Task와 명확한 인터페이스는 각 기능을 개별적으로 테스트하기 쉽게 한다.

○ 코드 재사용성 향상 (Code Reuse)

- 모듈화 수준이 높고 다른 모듈과의 의존성이 낮은 코드는 다른 프로젝트에서도 쉽게 재사용할 수 있다.

○ 시스템 효율성 향상 (Improved Efficiency)

- RTOS 기반 프로그램은 Event-Driven 방식으로 설계할 수 있다. 따라서 아직 발생하지 않은 이벤트를 계속 확인하는 Polling 방식에 CPU 시간을 낭비하지 않아도 된다.
- Polling 방식

```
while(1) {  
    센서 확인  
    버튼 확인  
    통신 확인  
    ...  
}
```

실시간 운영체제

□ Why Use an RTOS? (계속)

○ Idle Time 활용

- 실행할 애플리케이션 Task가 없으면 FreeRTOS가 자동으로 생성하는 Idle Task가 실행된다.
 - 남은 CPU 처리 능력 측정
 - 백그라운드 작업 수행
 - 시스템 상태 점검
 - 프로세서를 저전력 모드로 전환

○ 전력 관리 (Power Management)

- RTOS를 사용하면 CPU가 불필요하게 동작하는 시간을 줄일 수 있으므로 프로세서가 저전력 상태에 머무를 수 있는 시간을 증가시킬 수 있다. 특히 FreeRTOS는 Tick-less Mode를 지원하여 CPU가 더 오랫동안 저전력 모드에 머무를 수 있도록 한다.



실시간 운영체제

□ Why Use an RTOS? (계속)

○ 유연한 인터럽트 처리 (Flexible Interrupt Handling)

- 인터럽트 서비스 루틴(ISR)을 가능한 한 짧게 유지하고 실제 처리는 별도의 Task로 넘길 수 있다. FreeRTOS에서는 애플리케이션 Task 또는 자동으로 생성되는 RTOS Daemon Task(Timer Task)로 처리를 연기할 수 있다.



○ 다양한 처리 요구사항 지원 (Mixed Processing Requirements)

- FreeRTOS에서는 하나의 시스템 안에서 다양한 실행 형태를 함께 구현할 수 있다. 또한 Task와 Interrupt의 Priority를 적절히 설정하여 Hard Real-Time과 Soft Real-Time 요구사항을 동시에 처리할 수 있다.
 - Periodic Processing – 주기적 처리
 - Continuous Processing – 연속 처리
 - Event-Driven Processing – 이벤트 기반 처리

실시간 운영체제

□ FreeRTOS Kernel 주요 기능

○ 핵심 기능

Preemptive Scheduling	선점형 스케줄링
Cooperative Scheduling	협력형 스케줄링
Time Slicing	동일 우선순위 Task 시간 분할
Task Priority	우선순위 기반 작업
Task Notification	Task간 경량 통신
Queue	Task간 데이터 전달
Binary Semaphore	동기화
Counting Semaphore	자원 또는 이벤트 개수 관리
Mutex	상호 배제
Recursive Mutex	재귀적 Mutex
Software Timer	소프트웨어 타이머
Event Group	비트 단위 이벤트 관리

FreeRTOS 커널 배포판

Introduction

□ 본 장의 구성

- FreeRTOS 커널의 파일과 디렉터리 구조
 - FreeRTOS 디렉터리 구조의 상위 수준 구성
 - 특정 FreeRTOS 프로젝트에 필요한 소스 파일
 - 데모 애플리케이션(Demo Applications)
 - 새로운 FreeRTOS 프로젝트를 만드는 방법

FreeRTOS 배포판 이해

□ FreeRTOS Port

○ FreeRTOS가 지원하는 “컴파일러 + 프로세서 아키텍처”의 각 조합

- FreeRTOS는 약 20종의 서로 다른 컴파일러로 빌드할 수 있으며, 40종이 넘는 프로세서 아키텍처에서 실행할 수 있다.

□ FreeRTOS 빌드

○ 개요

- FreeRTOS는 원래 단일 스레드로 동작하는 Bare-Metal 애플리케이션에 멀티태스킹 기능을 제공하는 라이브러리이다.
- FreeRTOS는 여러 개의 C 소스 파일로 제공되며, 일부 소스 파일은 모든 Port에서 공통으로 사용하고, 일부는 특정 Port에만 사용된다.
- 필요한 소스 파일을 애플리케이션 프로젝트에 포함하여 함께 빌드하면 FreeRTOS API를 사용할 수 있다.

FreeRTOS 배포판 이해

□ FreeRTOSConfig.h

○ 기능

- FreeRTOSConfig.h는 FreeRTOS 커널의 동작을 애플리케이션에 맞게 설정하는 구성 헤더 파일이다.
- 이 파일에 정의된 상수들을 이용해 커널 기능을 선택하고 조정합니다.

○ 중요!!

- 소스 파일에서 FreeRTOSConfig.h를 직접 include하지 않으며, 대신 FreeRTOS.h를 include하면 FreeRTOS.h가 적절한 시점에 FreeRTOSConfig.h를 포함하게 된다.
- 예시
 - configUSE_PREEMPTION 상수는 FreeRTOS가 협력형(Co-operative) 스케줄링을 사용할지, 선점형(Pre-emptive) 스케줄링을 사용할지를 설정한다.
 - FreeRTOSConfig.h는 특정 애플리케이션에 맞춘 설정 파일이므로 FreeRTOS 커널 소스 디렉터리가 아니라 애플리케이션 프로젝트의 디렉터리에 두는 것이 적절하다.
 - 메인 FreeRTOS 배포판에는 각 FreeRTOS Port별 데모 애플리케이션과 FreeRTOSConfig.h가 포함되어 있다. 처음부터 새로 작성하기보다는, 사용하려는 Port의 데모 프로젝트에 포함된 FreeRTOSConfig.h를 복사한 후 필요한 설정을 수정하는 방법을 권장한다.
 - FreeRTOSConfig.h에 모든 설정 상수를 반드시 작성할 필요는 없으며, 생략된 많은 설정 항목은 기본값(default value)을 사용합니다.

실시간 운영체제

□ 공식 배포판

○ Git URL을 통한 다운로드 방법

- `git clone https://github.com/FreeRTOS/FreeRTOS.git --recurse-submodules`
- `git clone git@github.com:FreeRTOS/FreeRTOS.git --recurse-submodules`

○ 메인 배포판의 상위 디렉터리 구조

FreeRTOS

- └─ Source
- └─ Demo

FreeRTOS 커널 소스 파일

Port별로 미리 구성된 FreeRTOS 커널 데모 프로젝트

FreeRTOS-Plus

- └─ Source
- └─ Demo

기타 FreeRTOS 및 생태계 라이브러리 소스 코드

기타 FreeRTOS 및 생태계 라이브러리 데모 프로젝트

- 배포판에는 FreeRTOS 커널 소스 파일이 한 벌만 들어 있으며, 모든 데모 프로젝트는 FreeRTOS/Source 디렉터리에서 커널 소스 파일을 찾도록 구성되어 있다. 따라서 이 디렉터리 구조를 임의로 변경하면 데모 프로젝트가 빌드되지 않을 수 있다.

실시간 운영체제

□ 모든 Port에 공통인 FreeRTOS 소스 파일

○ 파일명 및 역할

- tasks.c와 list.c는 FreeRTOS 커널의 핵심 기능을 구현하므로 항상 필요하다. 이 파일들은 FreeRTOS/Source 디렉터리에 있으며, 같은 디렉터리에는 다음과 같은 선택적으로 사용할 수 있는 소스 파일도 있다.

파일명	역할	사용 여부
tasks.c	FreeRTOS 커널 핵심 Task 관리 기능	항상 필요
list.c	커널 내부 리스트 자료구조 및 핵심 기능	항상 필요
queue.c	Queue와 Semaphore 서비스 제공	거의 항상 필요
timers.c	Software Timer 기능 제공	Software Timer 사용 시
event_groups.c	Event Group 기능 제공	Event Group 사용 시
stream_buffer.c	Stream Buffer와 Message Buffer 기능 제공	해당 Buffer 사용 시
croutine.c	Co-routine 기능 구현	선택 사항; 현재 유지보수 중단

- Co-routine은 매우 작은 마이크로컨트롤러를 위해 설계된 기능으로 현재는 거의 사용되지 않는다. 더 이상 유지보수되지 않으며 새로운 설계에서는 사용을 권장하지 않는다.

실시간 운영체제

□ 모든 Port에 공통인 FreeRTOS 소스 파일 (계속)

○ 파일명 및 역할 (계속)

FreeRTOS	
└─ Source	
├─ tasks.c	항상 필요
├─ list.c	항상 필요
├─ queue.c	거의 항상 필요
├─ timers.c	선택 사항
├─ event_groups.c	선택 사항
├─ stream_buffer.c	선택 사항
└─ croutine.c	선택 사항 / 유지보수 중단

- ZIP 배포판에 사용되는 파일 이름은 다른 프로젝트에서 이미 사용하는 이름과 충돌할 수 있다. 필요하다면 사용자가 파일 이름을 변경할 수 있지만, FreeRTOS 공식 배포판 자체의 파일 이름은 기존 프로젝트와 FreeRTOS 인식 개발 도구의 호환성을 위해 변경할 수 없다.

실시간 운영체제

□ Port별 FreeRTOS 소스 파일

○ 구성

- FreeRTOS/Source/portable 디렉터리에는 특정 FreeRTOS Port에 필요한 소스 파일이 들어 있다. 이 디렉터리는 먼저 컴파일러별로, 그다음 프로세서 아키텍처별로 구성된다.
- 예를 들어 compiler라는 컴파일러와 architecture라는 프로세서 아키텍처를 사용하는 경우, 공통 FreeRTOS 소스 파일과 함께 다음 디렉터리의 소스 파일을 빌드해야 한다.

FreeRTOS/Source/portable/[compiler]/[architecture]

- FreeRTOS에서는 Heap 메모리 할당 기능도 Portable Layer의 일부로 취급한다. configSUPPORT_DYNAMIC_ALLOCATION을 0으로 설정했다면 프로젝트에 Heap 메모리 할당 구현을 포함하지 않는다.
- 동적 메모리 할당을 사용하는 경우에는 FreeRTOS/Source/portable/MemMang 디렉터리에 제공되는 Heap 구현 중 하나를 프로젝트에 포함하거나, 애플리케이션에서 자체 구현을 제공해야 한다.

FreeRTOS가 제공하는 예제 Heap 메모리 할당 구현은 한 프로젝트에 둘 이상 포함하지 않는다.

실시간 운영체제

□ Port별 FreeRTOS 소스 파일 (계속)

○ 구성 (계속)

```
FreeRTOS
├── Source
│   ├── portable
│   │   ├── MemMang
│   │   │   └── 대체 Heap 메모리 할당 구현
│   │   ├── [compiler 1]
│   │   │   ├── [architecture 1]
│   │   │   ├── [architecture 2]
│   │   │   └── [architecture 3]
│   │   └── [compiler 2]
│   │       ├── [architecture 1]
│   │       └── [architecture 2]
│   └── ...
```

실시간 운영체제

□ Include 경로

○ FreeRTOS 빌드를 위한 컴파일러 Include Path 지정

- 1. FreeRTOS 커널의 핵심 헤더 파일 경로: FreeRTOS/Source/include
- 2. 사용하는 Port의 소스 파일 경로: FreeRTOS/Source/portable/[compiler]/[architecture]
- 3. 올바른 FreeRTOSConfig.h가 위치한 디렉터리

□ 헤더 파일

○ 헤더 파일 include 방법

- FreeRTOS API를 사용하는 소스 파일은 먼저 FreeRTOS.h를 include하고, 그 다음 사용하려는 API 함수의 프로토타입이 선언된 헤더 파일을 include해야 한다.

```
#include "FreeRTOS.h"
#include "task.h"      // Task API 사용 시
#include "queue.h"     // Queue API 사용 시
#include "semphr.h"    // Semaphore / Mutex API 사용 시
```

- API 종류에 따라 task.h, queue.h, semphr.h, timers.h, event_groups.h, stream_buffer.h, message_buffer.h 또는 croutine.h를 사용한다. FreeRTOSConfig.h는 FreeRTOS.h가 자동으로 포함하므로 직접 include하지 않는다.

데모 애플리케이션

□ 개요

○ 각 FreeRTOS Port별 데모 애플리케이션 제공

- 작성 당시에는 컴파일 오류나 경고 없이 바로 빌드되도록 구성되어 있다.
- Cross platform support

FreeRTOS는 Windows, Linux, macOS 환경과 다양한 Embedded/일반 Toolchain에서 개발되고 테스트된다. 다만 Toolchain 버전 차이나 테스트 누락으로 인해 빌드 오류가 발생할 수 있다.

○ 제공 목적

- FreeRTOS API 사용 방법 시연
- FreeRTOS 커널 구현에 대한 Stress Test 수행
- 실제 애플리케이션을 개발할 때 출발점으로 활용
- 최소한의 설정과 사전 지식만으로 바로 실험할 수 있는 환경 제공
- 필요한 파일과 컴파일러 옵션이 올바르게 설정된 동작 가능한 프로젝트 예시 제공

데모 애플리케이션

□ 개요 (계속)

○ 구성

- 각 Demo Project는 FreeRTOS/Demo 아래의 고유한 하위 디렉터리에 위치하며, 디렉터리 이름은 해당 데모가 어떤 Port를 대상으로 하는지 나타낸다.
- 각 데모의 안내 페이지에서는 프로젝트 파일의 위치, 대상 하드웨어 또는 에뮬레이터, 하드웨어 설정 방법, 빌드 방법, 예상 동작 등을 확인할 수 있다.
- 모든 데모 프로젝트는 FreeRTOS/Demo/Common/Minimal에 구현된 “common demo tasks” 중 일부를 생성한다. 이 Task들은 실제 응용 기능을 제공하기 위한 것이 아니라 FreeRTOS API 사용법을 보여주고 커널 Port를 테스트하기 위해 존재한다.
- 많은 데모 프로젝트는 간단한 Blinky 형태의 시작 프로젝트로도 구성할 수 있으며, 일반적으로 두 개의 RTOS Task와 하나의 Queue를 생성한다.
- 모든 데모 프로젝트에는 main() 함수가 들어 있는 main.c 파일이 있으며, main()은 FreeRTOS 커널을 시작하기 전에 Demo Application의 Task들을 생성한다.

FreeRTOS

└ Demo

├ [Demo x] demo x 프로젝트 파일

├ [Demo y] demo y 프로젝트 파일

├ [Demo z] demo z 프로젝트 파일

└ Common 모든 데모에서 공통으로 빌드하는 파일

FreeRTOS 프로젝트 만들기

□ 제공된 Demo Project 수정하기

○ 개요

- 각 FreeRTOS Port에는 미리 구성된 Demo Application이 있으므로, 새로운 프로젝트는 기존 Demo Project를 수정하여 시작하는 것이 권장된다. 이렇게 하면 필요한 소스 파일, Interrupt Handler, Compiler Option을 올바르게 설정할 가능성이 높아진다.

○ 기존 Demo Project에서 새 애플리케이션을 만드는 절차

이 과정을 마치면 FreeRTOS에 필요한 소스 파일과 설정은 갖추고 있지만 아직 애플리케이션 기능은 정의하지 않은 깨끗한 프로젝트가 만들어진다.

1. 제공된 Demo Project를 열고 정상적으로 빌드되고 실행되는지 확인한다.
2. Demo/Common 디렉터리에 있는 Demo Task 구현 소스 파일을 제거한다.
3. main()에서 prvSetupHardware()와 vTaskStartScheduler()를 제외한 함수 호출을 제거한다.
4. 프로젝트가 여전히 정상적으로 빌드 되는지 확인한다.

○ 새 main() 함수의 기본 템플릿

```
int main( void ) {
    prvSetupHardware(); /* 필요한 하드웨어 초기화 */

    /* --- APPLICATION TASKS CAN BE CREATED HERE --- */

    vTaskStartScheduler(); /* 생성한 Task 실행 시작 */

    for( . ;; ); /* Heap 부족으로 Scheduler를 시작하지 못한 경우에만 도달 */

    return 0;
}
```

FreeRTOS 프로젝트 만들기

□ 처음부터 새 프로젝트 만들기

○ 개요

- 가능하면 기존 Demo Project를 기반으로 새 프로젝트를 만드는 것이 권장된다. 부득이하게 처음부터 프로젝트를 구성해야 한다면 다음 절차를 따른다.
- 기존 Demo Project에서 새 애플리케이션을 만드는 절차
 1. 선택한 Toolchain에서 아직 FreeRTOS 소스 파일을 포함하지 않은 새 프로젝트를 만든다.
 2. 새 프로젝트가 정상적으로 빌드되고 Target Hardware로 다운로드되며 실행되는지 확인한다.
 3. 기본 프로젝트가 정상 동작한다는 것을 확인한 후 FreeRTOS에 필요한 소스 파일을 추가한다.
 4. 사용 Port의 Demo Project에 포함된 FreeRTOSConfig.h를 새 프로젝트 디렉터리로 복사한다.
 5. FreeRTOS/Source/include, FreeRTOS/Source/portable/[compiler]/[architecture], FreeRTOSConfig.h가 있는 디렉터리를 Include Path에 추가한다.
 6. 관련 Demo Project의 Compiler Setting을 복사한다.
 7. 필요한 FreeRTOS Interrupt Handler를 설치한다. 해당 Port의 안내 페이지와 Demo Project를 참고한다.

FreeRTOS 프로젝트 만들기

□ 처음부터 새 프로젝트 만들기 (계속)

- 프로젝트에 포함해야 하는 FreeRTOS 소스 파일과 위치

파일명	저장 위치
queue.c	FreeRTOS/Source
list.c	FreeRTOS/Source
timers.c	FreeRTOS/Source
event_groups.c	FreeRTOS/Source
stream_buffer.c	FreeRTOS/Source
Port별 모든 C/Assembler 파일	FreeRTOS/Source/portable/[compiler]/[architecture]
heap_n.c (n = 1, 2, 3, 4, 5)	FreeRTOS/Source/portable/MemMang

➤ Heap 메모리 참고

configSUPPORT_DYNAMIC_ALLOCATION이 0이면 Heap 메모리 할당 구현을 프로젝트에 포함하지 않는다. 동적 할당을 사용한다면 heap_n.c 중 하나 또는 사용자가 직접 만든 Heap 구현을 포함해야 한다.

데이터 형식과 코딩 스타일 가이드

□ 데이터 형식

각 FreeRTOS Port에는 고유한 portmacro.h 헤더 파일이 있으며, 이 파일에는 Port별 데이터 형식 정의가 포함된다. 대표적으로 TickType_t와 BaseType_t가 있다.

○ TickType_t

- FreeRTOS는 Tick Interrupt라고 하는 주기적 인터럽트를 사용한다. FreeRTOS 애플리케이션이 시작된 이후 발생한 Tick Interrupt의 횟수를 Tick Count라고 하며, 시간을 측정하는 기준으로 사용한다.
- 두 Tick Interrupt 사이의 시간을 Tick Period라고 하며, FreeRTOS의 시간 값은 일반적으로 Tick Period의 배수로 표현된다. TickType_t는 Tick Count를 저장하고 시간 값을 지정할 때 사용하는 데이터 형식이다.
- TickType_t는 FreeRTOSConfig.h의 configTICK_TYPE_WIDTH_IN_BITS 설정에 따라 unsigned 16-bit, 32-bit 또는 64-bit 형식이 될 수 있다. 이 설정은 아키텍처에 따라 달라지며 각 FreeRTOS Port가 설정값의 유효성을 검사한다.
- 8-bit 또는 16-bit 아키텍처에서는 16-bit TickType_t가 효율을 높일 수 있지만, FreeRTOS API에서 지정할 수 있는 최대 Block Time이 크게 제한된다. 32-bit 또는 64-bit 아키텍처에서는 16-bit TickType_t를 사용할 이유가 없다.
- 기존의 configUSE_16_BIT_TICKS 설정은 32-bit를 초과하는 Tick Count를 지원하기 위해 configTICK_TYPE_WIDTH_IN_BITS로 대체되었다. 새로운 설계에서는 configTICK_TYPE_WIDTH_IN_BITS 사용을 권장한다.

데이터 형식과 코딩 스타일 가이드

□ 데이터 형식 (계속)

○ TickType_t (계속)

설정값	8-bit 아키텍처	16-bit 아키텍처	32-bit 아키텍처	64-bit 아키텍처
TICK_TYPE_WIDTH_16_BITS	uint16_t	uint16_t	uint16_t	N/A
TICK_TYPE_WIDTH_32_BITS	uint32_t	uint32_t	uint32_t	N/A
TICK_TYPE_WIDTH_64_BITS	N/A	N/A	uint64_t	uint64_t

○ BaseType_t

- BaseType_t는 해당 아키텍처에서 가장 효율적인 데이터 형식으로 정의된다. 일반적으로 64-bit 아키텍처에서는 64-bit, 32-bit 아키텍처에서는 32-bit, 16-bit 아키텍처에서는 16-bit, 8-bit 아키텍처에서는 8-bit 형식을 사용한다.
- BaseType_t는 반환값의 종류가 많지 않은 함수의 Return Type이나 pdTRUE/pdFALSE와 같은 Boolean 값에 주로 사용된다.

데이터 형식과 코딩 스타일 가이드

□ 변수 이름

FreeRTOS에서는 변수 이름 앞에 데이터 형식을 나타내는 접두사(prefix)를 붙인다.

접두사	의미/예시
c	char
s	int16_t (short)
l	int32_t (long)
x	BaseType_t 또는 구조체, Task Handle, Queue Handle 등 비표준 형식
u	unsigned 형식에 추가
p	pointer 형식에 추가

- 예를 들어 uint8_t 변수에는 uc 접두사를 사용하고, char* 형식의 포인터 변수에는 pc 접두사를 사용한다.

```
uint8_t ucValue;  
char *pcString;
```

데이터 형식과 코딩 스타일 가이드

☐ 함수 이름

함수 이름의 접두사는 “반환 형식”과 “함수가 정의된 소스 파일”을 함께 나타낸다.

예	의미
vTaskPrioritySet()	void를 반환하며 tasks.c에 정의
xQueueReceive()	BaseType_t를 반환하며 queue.c에 정의
pvTimerGetTimerID()	void*를 반환하며 timers.c에 정의
prv...	파일 내부에서만 사용하는 Private 함수 접두사

☐ 포매팅

일부 Demo Application에서는 Tab 한 개를 Space 네 칸으로 설정하여 사용한다. 현재 FreeRTOS Kernel 소스 코드는 더 이상 Tab 문자를 사용하지 않는다.

데이터 형식과 코딩 스타일 가이드

□ 매크로 이름

대부분의 FreeRTOS 매크로는 대문자로 작성하며, 매크로가 어느 파일에서 정의되었는지를 나타내는 소문자 접두사를 앞에 붙인다.

접두사	정의 위치	예
port	portable.h 또는 portmacro.h	portMAX_DELAY
task	task.h	taskENTER_CRITICAL()
pd	projdefs.h	pdTRUE
config	FreeRTOSConfig.h	configUSE_PREEMPTION
err	projdefs.h	errQUEUE_FULL

Semaphore API는 거의 대부분 매크로로 구현되어 있지만, 매크로 이름 규칙이 아니라 함수 이름 규칙을 따른다.

FreeRTOS 소스 코드 전반에서 자주 사용하는 공통 매크로는 다음과 같다.

Macro	Value
pdTRUE	1
pdFALSE	0
pdPASS	1
pdFAIL	0

데이터 형식과 코딩 스타일 가이드

□ 과도한 Type Casting을 사용하는 이유

FreeRTOS 소스 코드는 매우 다양한 컴파일러에서 빌드된다. 컴파일러마다 Warning을 발생시키는 조건과 Casting을 요구하는 방식이 서로 다르다.

그 결과 FreeRTOS 소스 코드에는 일반적인 코드보다 많은 Type Casting이 포함되어 있다. 이는 여러 컴파일러에서 일관되게 빌드하고 불필요한 Warning을 줄이기 위한 호환성 목적의 선택이다.

힙(Heap) 메모리 관리

개요

□ Introduction

○ 선수지식

- FreeRTOS를 사용하려면 C프로그래밍에 능숙해야 한다. 따라서 이 장에서는 독자가 다음 개념을 이미 알고있다고 가정한다.
 - C 프로젝트를 빌드할 때의 컴파일(compile) 단계와 링크(link) 단계
 - Stack과 Heap의 개념
 - 표준 C라이브러리의 malloc()과 free()함수

○ 범위

- 이 장에서는 다음 내용을 다룬다.
 - FreeRTOS가 RAM을 할당하는 시점
 - FreeRTOS가 제공하는 다섯 가지 예제 메모리 할당 방식
 - 애플리케이션에 적합한 메모리 할당 방식을 선택하는 방법

개요

□ Introduction(계속)

○ 정적 메모리 할당과 동적 메모리 할당 전환

- 이후장에서는 Task, Queue, Semaphore, Event Group과 같은 Kernel Object를 소개한다. 이러한 객체를 저장하는 데 필요한 RAM은 컴파일 시점에 정적으로 할당하거나 실행 중에 동적으로 할당할 수 있다.
 - Dynamic Allocation : 설계와 사전 계획 부담을 줄이고 API를 단순화하며 RAM 사용량을 최소화할 수 있다.
 - Static Allocation : 실행 시점의 동작이 더 결정적이며 메모리 할당 실패를 처리할 필요가 없고 Heap Fragmentation의 위험을 제거한다.
- Heap Fragmentation은 전체 여유 메모리의 양은 충분하지만 필요한 크기의 연속된 메모리 블록을 확보할 수 없는 상태를 의미한다.
- 정적으로 할당된 메모리를 사용하여 Kernel Object를 생성하는 FreeRTOS API는 FreeRTOSConfig.h에서 configSUPPORT_STATIC_ALLOCATION을 1로 설정한 경우에만 사용할 수 있다.
- 동적으로 할당된 메모리를 사용하여 Kernel Object를 생성하는 API는 configSUPPORT_DYNAMIC_ALLOCATION을 1로 설정하거나 해당 상수를 정의하지 않은 경우에 사용할 수 있다. 두 상수를 동시에 1로 설정하는 것도 허용한다.

개요

□ Introduction(계속)

○ 동적 메모리 할당 사용

- 동적 메모리 할당은 FreeRTOS나 멀티태스킹에만 해당하는 것이 아니라 일반적인 C프로그래밍 개념이다. FreeRTOS에서는 Kernel Object를 동적으로 생성할 수 있기 때문에 이 개념이 중요하다.
- 일반 목적의 C라이브러리 Malloc()과 free()가 임베디드 시스템에 적합하지 않을 수 있는 이유는 다음과 같다.
 - 소형 임베디드 시스템에서는 함수 자체를 제공하지 않는 경우가 있다.
 - 구현 코드의 크기가 상대적으로 커서 귀중한 Code Space를 많이 사용할 수 있다.
 - 대부분 Thread-Safe하지 않다.
 - 결정적(deterministic)이지 않다. 함수 호출마다 실행 시간이 달라질 수 있다.
 - Heap Fragmentation이 발생할 수 있다.
 - Linker 설정을 복잡하게 만들 수 있다.
 - Heap영역이 다른 변수의 메모리 영역까지 증가하도록 잘못 설정되면 디버깅하기 어려운 오류가 발생할 수 있다.

실시간 개요

□ Introduction(계속)

○ 동적 메모리 할당 방식의 선택

- 초기의 FreeRTOS는 서로 다른 크기의 메모리 블록 풀을 컴파일 시점에 미리 만들어 두고 메모리 할당 함수가 그 블록을 반환하는 Memory Pool 방식을 사용했다.
- Block Allocation은 실시간 시스템에서 흔히 사용하는 방식이지만, 매우 작은 임베디드 시스템에서 RAM을 비효율 적으로 사용하는 문제가 있었기 때문에 FreeRTOS에서 제거되었다.
- 현재 FreeRTOS는 메모리 할당 기능을 Core Code가 아니라 Portable Layer의 일부로 취급한다. 임베디드 시스템마다 동적 메모리 할당과 타이밍 요구사항이 다르므로 하나의 알고리즘이 모든 애플리케이션에 적합할 수 없기 때문이다. 또한 메모리 할당을 Core에서 분리하면 애플리케이션 개발자가 필요에 맞는 자체 구현을 제공할 수 있다.
- FreeRTOS가 RAM을 필요로 할 때에는 malloc() 대신 pvPortMalloc()을 호출하고, 기존에 할당한 RAM을 반환할 때에는 free() 대신 vPortFree()를 호출한다. pvPortMalloc()은 표준C의 malloc()과 동일한 프로토타입을 가지며, vPortFree()도 free()와 동일한 프로토타입을 가진다.

개요

□ Introduction(계속)

○ 동적 메모리 할당 방식의 선택(계속)

- `pvPortMallco()` 과 `vPortFree()`는 Public Function이므로 애플리케이션 코드에서도 직접 호출할 수 있다.
- FreeRTOS는 `pvPortmalloc()` 과 `vPortFree()`의 예제 구현 다섯 가지를 제공한다. 애플리케이션은 이 중 하나를 사용하거나 자체 구현을 제공할 수 있다.

FreeRTOS/Source/portable/MemMang/

- ├─ heap_1.c
- ├─ heap_2.c
- ├─ heap_3.c
- ├─ heap_4.c
- └─ heap_5.c

메모리 할당 방식

□ 예제 메모리 할당 방식

○ Heap_1

- 소형 전용 임베디드 시스템에서는 FreeRTOS Scheduler를 시작하기 전에 모든 Task와 Kernel Object를 생성하는 경우가 흔하다. 이 경우 동적 메모리 할당은 실시간 기능이 시작되기 전에만 발생하고, 할당된 메모리는 애플리케이션의 수명동안 계속 유지된다.
- 따라서 이런 시스템에서는 결정성이나 Fragmentation과 같은 복잡한 메모리 관리 문제보다 Code Size와 단순성을 우선할 수 있다.
- Heap_1_c는 매우 단순한 pvPortMalloc()을 구현하지만 vPortFree()는 구현하지 않는다. Task나 다른 Kernel Object를 삭제하지 않는 애플리케이션에서 heap_1을 사용할 수 있다.
- 일부 상업적으로 중요한 시스템이나 Safety_Critical 시스템에서는 일반적으로 동적 메모리 할당을 금지하지만, heap_1은 사용할 가능성이 있다. 이러한 시스템에서 동적 할당을 제한하는 주된 이유는 비결정성, 메모리 단편화, 할당 실패에 대한 불확실성이다.
 - Heap_1의 핵심 특성 : heap_1은 항상 결정적이며 메모리 단편화가 발생하지 않는다.
- Heap_1의 pvPortMalloc()은 FreeRTOS Heap으로 단순한 unit8_t 배열을 호출될 때마다 작은 블록으로 나누어 할당한다. 배열의 크기는 FreeRTOSConfig.h의 configTOTAL_HEAP_SIZE가 바이트 단위로 결정된다.

메모리 할당 방식

□ 예제 메모리 할당 방식(계속)

○ Heap_1(계속)

- Heap을 정적으로 할당된 배열로 구현하기 때문에 FreeRTOS데이터 영역에 Heap배열이 포함되며, 결과적으로 FreeRTOS가 많은 RAM을 사용하는 것처럼 보일 수 있다.
- 동적으로 생성한 각 Task는 pvPortMalloc()을 두 번 호출한다. 첫 번째 호출은 Task Control Block(TCB)을 위한 메모리를 할당하고, 두 번째 호출은 Task Stack을 위한 메모리를 할당한다.

○ Task생성에 따라 heap_1 배열에서 RAM이 할당되는 과정

단계	상태/의미
A	Task를 생성하기 전 상태이며 배열 전체가 Free Space이다.
B	Task 한 개를 생성한 후의 상태이며 TCB와 Stack에 메모리가 할당된다.
C	Task 세 개를 생성한 후의 상태이며 각 Task의 TCB와 Stack이 순차적으로 Heap을 사용한다.

메모리 할당 방식

□ 예제 메모리 할당 방식(계속)

- Task생성에 따라 heap_1 배열에서 RAM이 할당되는 과정(Figuer 3.2)

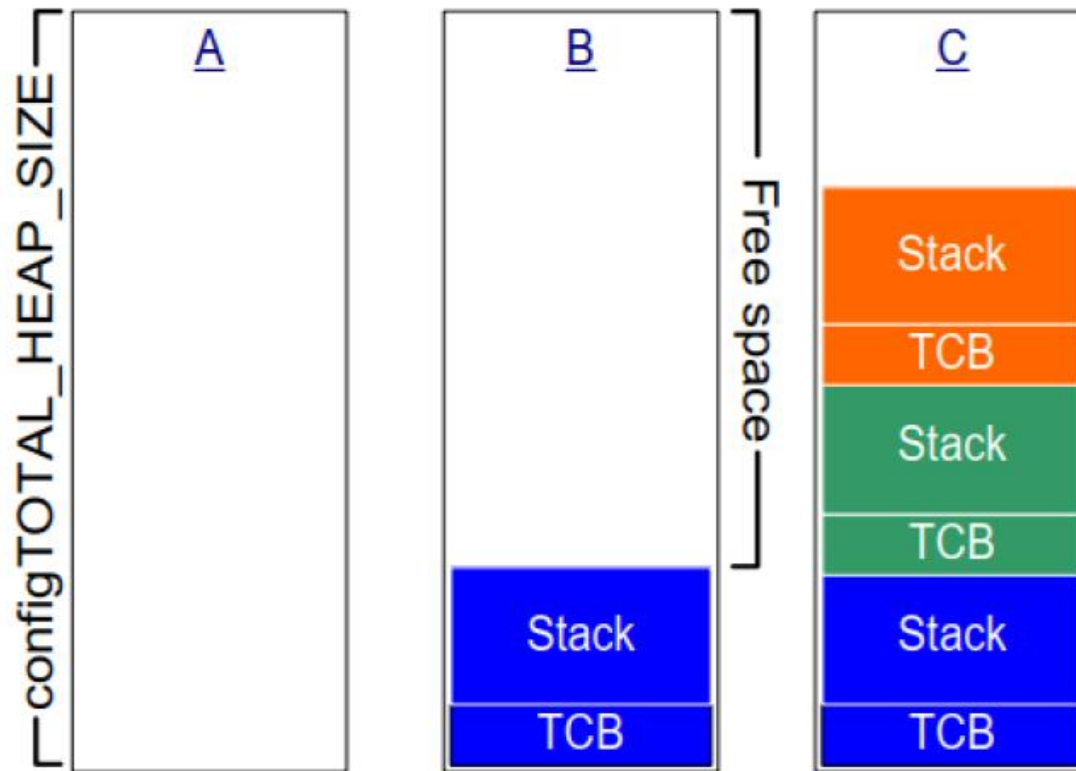


Figure 3.1 RAM being

allocated from the heap_1 array each time a task is created

메모리 할당 방식

□ 예제 메모리 할당 방식(계속)

○ Heap_2

- heap_2는 기능이 향상된 heap_4로 대체되었다. 현재 FreeRTOS 배포판에는 하위 호환성을 위해 남아 있지만 새로운 설계에서는 사용을 권장하지 않는다.
- heap_2.c도 configTOTAL_HEAP_SIZE가 정한 크기의 배열을 작은 블록으로 나누어 사용한다. heap_1과 달리 vPortFree()를 구현하며, 메모리 할당에는 Best-Fit Algorithm을 사용한다.
- Best-Fit Algorithm은 요청한 크기를 수용할 수 있는 Free Block 중 요청 크기에 가장 가까운 블록을 선택한다.
 - **Best-Fit 예** : Heap에 5바이트, 25바이트, 100바이트의 Free Block이 있고 pvPortMalloc()이 20바이트를 요청하면 25바이트 블록을 선택한다. 이 블록을 20바이트와 5바이트로 나누고 20바이트 블록의 포인터를 반환한다.
- 실제 heap_2는 Heap 영역 내부에 Block Size 정보를 저장하므로, 위 예의 두 분할 블록의 실제 합은 25바이트보다 작다.
- heap_2는 heap_4와 달리 서로 인접한 Free Block을 하나의 큰 블록으로 병합하지 않는다. 따라서 heap_4보다 Fragmentation에 취약하다. 단, 할당하고 해제하는 블록의 크기가 항상 같다면 Fragmentation은 문제가 되지 않는다.

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Task 생성과 삭제에 따른 heap_2의 RAM 할당 및 해제

단계	상태/의미
A	Task 세 개를 할당한 상태이며 배열 상단에 큰 Free Block이 남아 있다.
B	Task 하나를 삭제한 상태이다. 삭제한 Task의 TCB와 Stack이 사용하던 위치에 작은 Free Block 두 개가 생긴다.
C	새 Task를 생성한 상태이다. xTaskCreate()가 pvPortMalloc()을 두 번 호출하여 TCB와 Stack을 할당한다. TCB는 크기가 항상 같으므로 삭제된 Task의 TCB 공간을 재사용한다. 새 Stack 크기가 이전 Stack과 같으면 그 공간도 재사용한다. 상단의 큰 Free Block은 그대로 남는다.

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

- Task 생성과 삭제에 따른 heap_2의 RAM 할당 및 해제(Figuer 3.2)

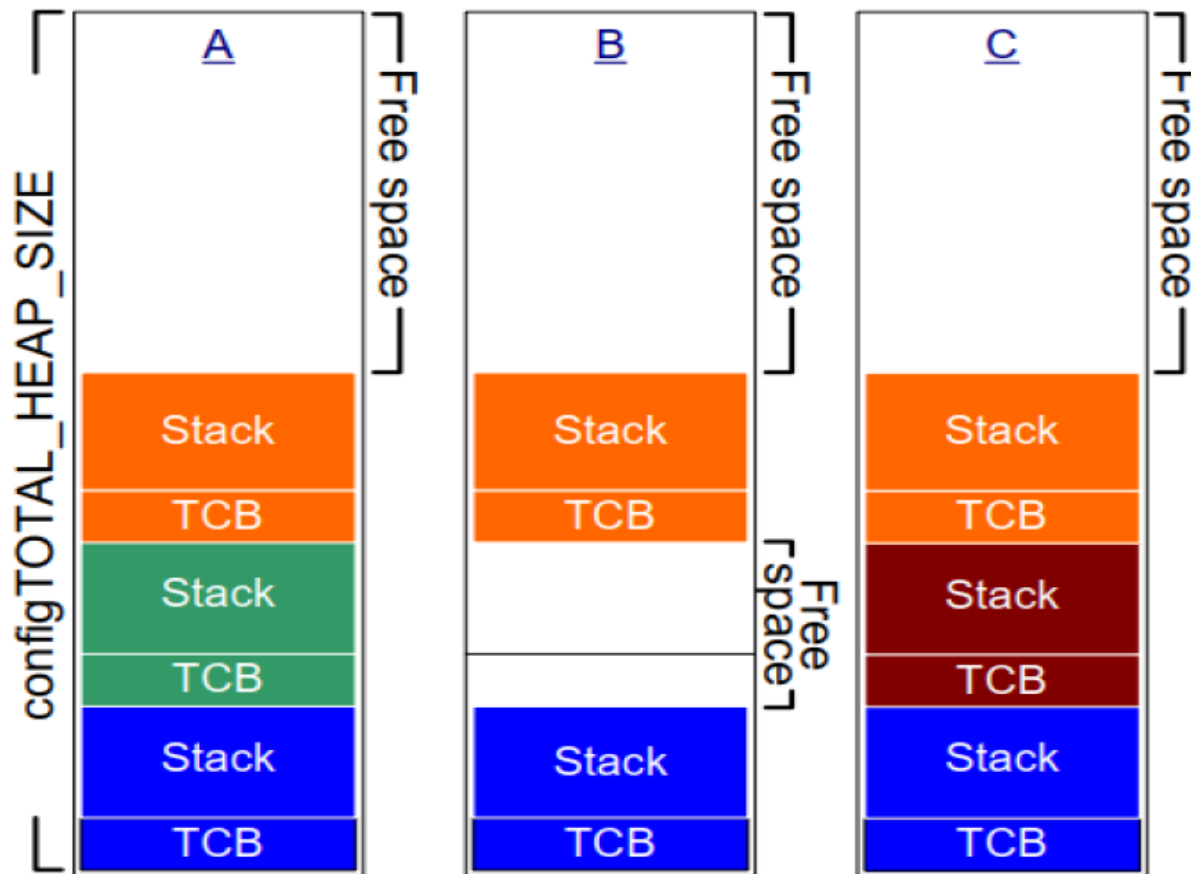


Figure 3.2 RAM being

allocated and freed from the heap_2 array as tasks are created and deleted

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_3

- Heap_3.c는 표준라이브러리 malloc()과 free()를 사용한다. 따라서 Heap크기는 Linker 설정으로 결정하며 configTOTAL_HEAP_SIZE는 사용하지 않는다.
- Heap_3는 malloc()과 free()가 실행되는 동안 FreeRTOS Scheduler를 일시적으로 중단하여 두 함수를 Thread-Safety 하게 만든다. Thread Safety와 Scheduler Suspension은 Chapter 8에서 다룬다.

○ Heap_4

- heap_4도 heap_1, heap_2와 마찬가지로 정적으로 할당한 배열을 작은 블록으로 나누어 사용하며 배열의 크기는 configTOTAL_HEAP_SIZE가 결정한다.
- heap_4는 First-Fit Algorithm을 사용한다. heap_2와 달리 인접한 Free Block을 하나의 큰 블록으로 병합(coalesce)하므로 Memory Fragmentation의 위험을 줄인다.
- First-Fit Algorithm은 요청한 크기를 수용할 수 있는 첫 번째 Free Block을 선택한다.
 - **First-Fit 예** : 배열 순서대로 5바이트, 200바이트, 100바이트의 Free Block이 있고 pvPortMalloc()이 20바이트를 요청하면 가장 먼저 조건을 만족하는 200바이트 블록을 선택한다. 이를 20바이트와 180바이트 블록으로 나누고 20바이트 블록을 반환한다.

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_4(계속)

- 실제 heap_4는 Heap 내부에 Block Size 정보를 저장하므로 위 예에서 분할된 두 블록의 실제 합은 200바이트보다 작다.
- 인접한 Free Block을 병합하는 특성 때문에 heap_4는 서로 다른 크기의 메모리 블록을 반복적으로 할당하고 해제하는 애플리케이션에 적합하다.

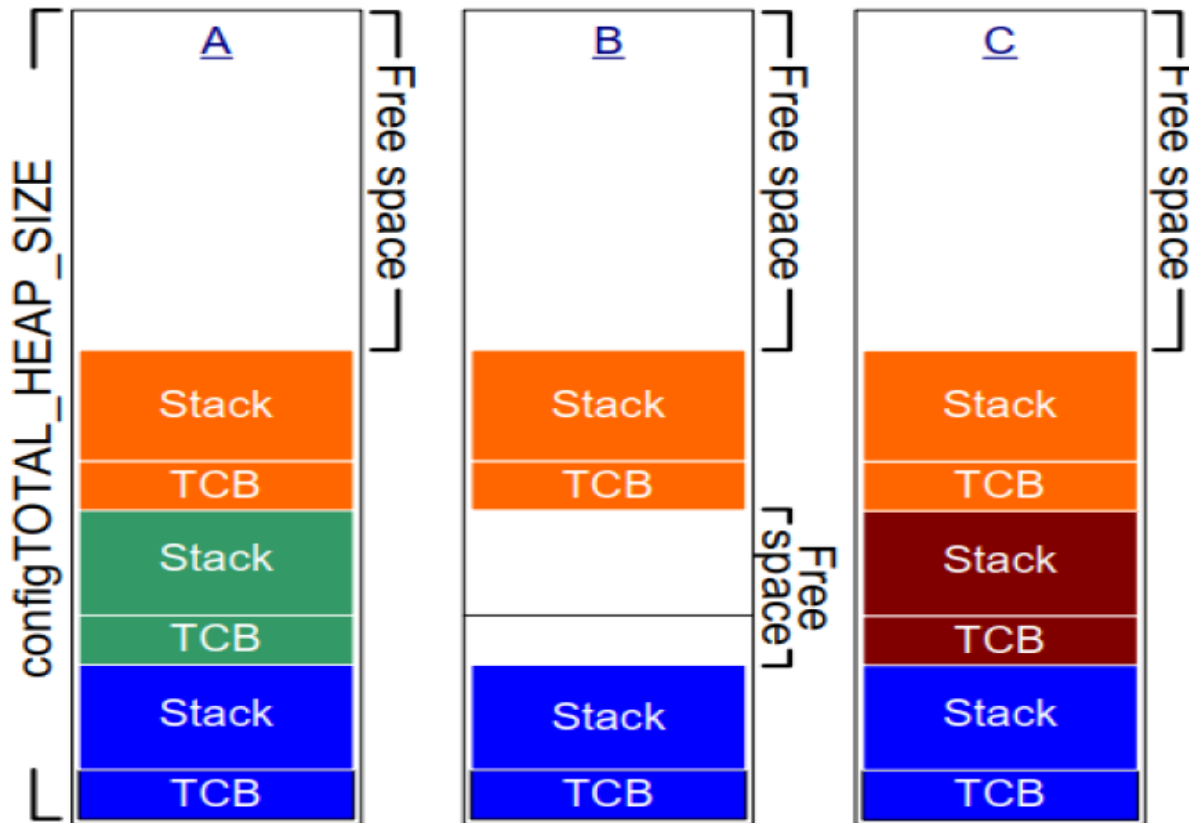
○ heap_4의 First-Fit 및 Free Block 병합 과정

단계	상태/의미
A	Task 세 개를 생성한 상태이며 배열 상단에 큰 Free Block이 남아 있다.
B	Task 하나를 삭제한다. heap_2와 달리 삭제된 Task의 TCB와 Stack 공간을 하나의 큰 Free Block으로 병합한다.
C	FreeRTOS Queue를 생성한다. xQueueCreate()가 pvPortMalloc()을 호출하고, 첫 번째로 충분히 큰 Free Block에서 Queue 메모리를 할당한다. 남은 공간은 다시 Free Block으로 유지한다.
D	애플리케이션 코드에서 pvPortMalloc()을 직접 호출한다. 사용자 할당 블록이 첫 번째 Free Block의 일부를 사용한다.
E	Queue를 삭제하면 Queue가 사용하던 메모리가 자동으로 해제되어 사용자 할당 블록의 양쪽에 Free Space가 생긴다.
F	사용자 할당 블록까지 해제하면 양쪽의 Free Space와 병합되어 하나의 더 큰 Free Block이 된다.

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ heap_4의 First-Fit 및 Free Block 병합 과정(Figuer 3.3)



allocated and freed from the heap_2 array as tasks are created and deleted

Figure 3.2 RAM being

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_5

- 소형 전용 임베디드 시스템에서는 FreeRTOS Scheduler를 시작하기 전에 모든 Task와 Kernel Object를 생성하는 경우가 흔하다. 이 경우 동적 메모리 할당은 실시간 기능이 시작되기 전에만 발생하고, 할당된 메모리는 애플리케이션의 수명동안 계속 유지된다.
- 따라서 이런 시스템에서는 결정성이나 Fragmentation과 같은 복잡한 메모리 관리 문제보다 Code Size와 단순성을 우선할 수 있다.
- Heap_1_c는 매우 단순한 pvPortMalloc()을 구현하지만 vPortFree()는 구현하지 않는다. Task나 다른 Kernel Object를 삭제하지 않는 애플리케이션에서 heap_1을 사용할 수 있다.
- 일부 상업적으로 중요한 시스템이나 Safety_Critical 시스템에서는 일반적으로 동적 메모리 할당을 금지하지만, heap_1은 사용할 가능성이 있다. 이러한 시스템에서 동적 할당을 제한하는 주된 이유는 비결정성, 메모리 단편화, 할당 실패에 대한 불확실성이다.
 - Heap_1의 핵심 특성 : heap_1은 항상 결정적이며 메모리 단편화가 발생하지 않는다.
- Heap_1의 pvPortMalloc()은 FreeRTOS Heap으로 단순한 unit8_t 배열을 호출될 때마다 작은 블록으로 나누어 할당한다. 배열의 크기는 FreeRTOSConfig.h의 configTOTAL_HEAP_SIZE가 바이트 단위로 결정된다.

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_1

- heap_5는 heap_4와 동일한 메모리 할당 알고리즘을 사용한다. 차이는 heap_4가 하나의 배열에서만 메모리를 할당하는 반면, heap_5는 서로 떨어져 있는 여러 메모리 영역을 하나의 Heap으로 결합할 수 있다는 점이다.
- FreeRTOS가 실행되는 시스템의 RAM이 Memory Map에서 하나의 연속된 영역으로 나타나지 않는 경우 heap_5가 유용하다.

○ 비교

방식	해제 지원	할당 방식/특징	주요 용도
heap_1	X	단순 순차 할당, 항상 결정적, Fragmentation 없음	객체를 생성한 뒤 삭제하지 않는 단순/안전 중심 시스템
heap_2	O	Best-Fit, 인접 Free Block 병합 없음	하위 호환성 목적, 새 설계에는 비권장
heap_3	O	표준 malloc()/free() 사용, Scheduler 중단으로 Thread-Safe 보장	표준 C Heap을 그대로 사용하려는 경우
heap_4	O	First-Fit, 인접 Free Block 병합	서로 다른 크기의 블록을 반복 할당/해제하는 일반적인 경우
heap_5	O	heap_4와 동일하나 여러 비연속 RAM 영역을 하나의 Heap으로 구성	RAM이 여러 구간으로 분리된 시스템

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_5 초기화 :vPortDefineHeapRegions() API

- vPortDefineHeapRegions()는 heap_5가 관리할 각 메모리 영역의 시작 주소와 크기를 지정하여 heap_5를 초기화한다.
- heap_5는 FreeRTOS가 제공하는 Heap 할당 방식 중 명시적 초기화가 필요한 유일한 방식이다. vPortDefineHeapRegions()를 호출하기 전에는 사용할 수 없으므로, 해당 호출 이전에는 Task, Queue, Semaphore 같은 Kernel Object를 동적으로 생성할 수 없다.

- Listing 3.1 vPortDefineHeapRegions() API 함수 프로토타입

```
void vPortDefineHeapRegions( const HeapRegion_t * const pxHeapRegions );
```

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_5 초기화 :vPortDefineHeapRegions() API(계속)

- vPortDefineHeapRegions()의 유일한 매개변수는 HeapRegion_t 구조체 배열이다. 각 구조체는 Heap의 일부가 될 메모리 블록 하나의 시작 주소와 크기를 정의하며, 배열 전체가 heap_5가 사용할 전체 Heap 공간을 정의한다.

- Listing 3.2 HeapRegion_t 구조체

```
typedef struct HeapRegion
{
    /* Heap 일부가 될 메모리 블록의 시작 주소 */
    uint8_t *pucStartAddress;

    /* 메모리 블록의 크기(바이트) */
    size_t xSizeInBytes;
} HeapRegion_t;
```

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

○ Heap_5 초기화 :vPortDefineHeapRegions() API(계속)

- pxHeapRegions는 HeapRegion_t 배열의 첫 번째 요소를 가리키는 포인터이다. 배열의 각 구조체는 Heap에 포함될 메모리 블록의 시작 주소와 크기를 정의한다.
- HeapRegion_t 구조체는 시작 주소가 낮은 순서대로 배열에 배치해야 한다. 가장 낮은 시작 주소를 가진 영역이 첫 번째 요소이고, 가장 높은 시작 주소를 가진 영역이 마지막 요소가 된다. 배열의 끝은 pucStartAddress가 NULL인 HeapRegion_t 구조체로 표시한다.

○ 세 RAM 영역을 사용하는 가상 Memory Map

단계	상태/의미
A	RAM1 64KB, RAM2 32KB, RAM3 32KB의 세 메모리 영역 전체가 존재하는 기본 Memory Map이다. 실행 코드는 별도의 Read-Only Memory에 있다고 가정한다.
B	Linker가 RAM1 일부를 일반 변수에 사용하고 RAM2, RAM3는 사용하지 않은 상태이다. heap_5는 RAM1의 남은 부분과 RAM2, RAM3를 사용할 수 있다.
C	RAM1에 ucHeap 배열을 배치하고, RAM2와 RAM3 전체를 더해 heap_5가 관리하는 안전한 Heap 영역을 구성한 상태이다.

메모리 할당 방식

□ 예제 메모리 할당 방식 (계속)

- 세 RAM 영역을 사용하는 가상 Memory Map(Figure 3.4)

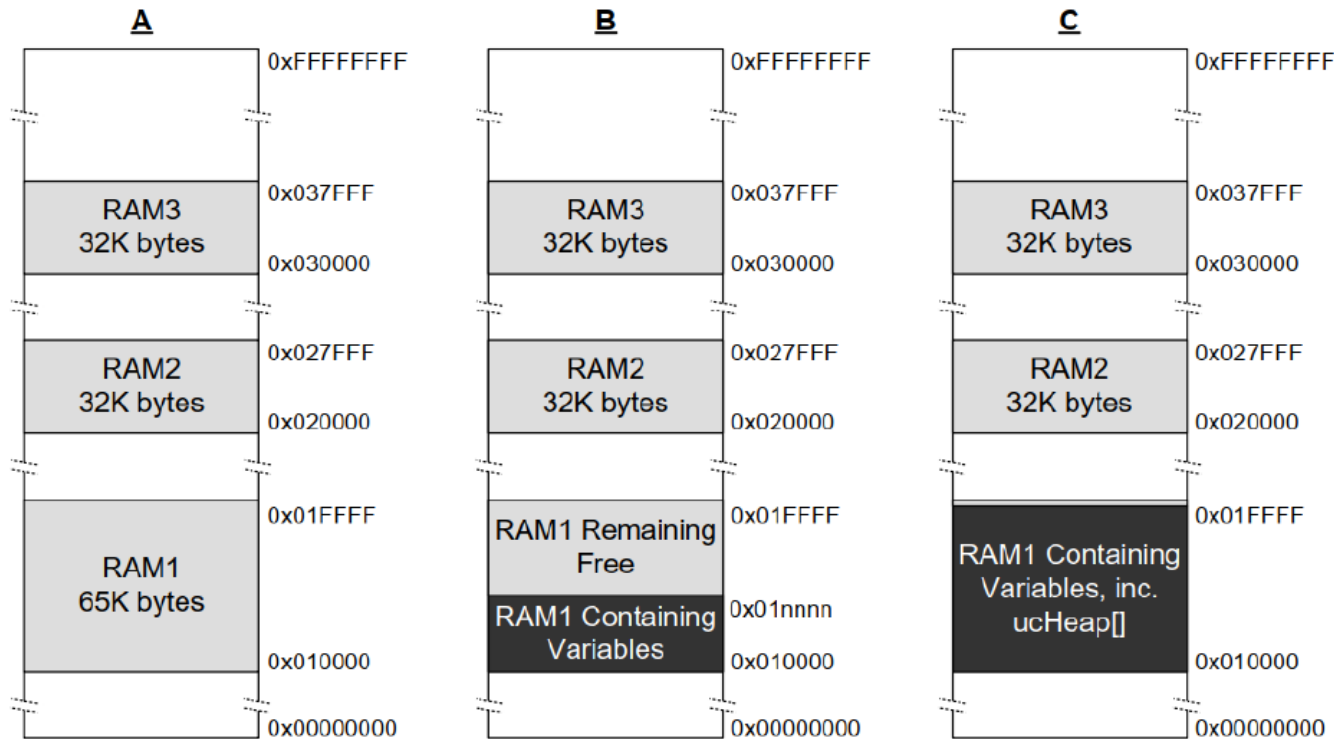


Figure 3.4 Memory Map

메모리 할당 방식

□ 예제 메모리 할당 방식

○ Heap_5 초기화 :vPortDefineHeapRegions() API(계속)

- 세 RAM 영역 전체를 Heap으로 정의하는 예는 다음과 같다.
- Listing 3.3 세 RAM 영역 전체를 기술하는 HeapRegion_t 배열

```
#define RAM1_START_ADDRESS ( ( uint8_t * ) 0x00010000 )
#define RAM1_SIZE          ( 64 * 1024 )
#define RAM2_START_ADDRESS ( ( uint8_t * ) 0x00020000 )
#define RAM2_SIZE          ( 32 * 1024 )
#define RAM3_START_ADDRESS ( ( uint8_t * ) 0x00030000 )
#define RAM3_SIZE          ( 32 * 1024 )

const HeapRegion_t xHeapRegions[] =
{
    { RAM1_START_ADDRESS, RAM1_SIZE },
    { RAM2_START_ADDRESS, RAM2_SIZE },
    { RAM3_START_ADDRESS, RAM3_SIZE },
    { NULL, 0 } /* 배열의 끝 */
};

int main( void )
{
    /* heap_5 초기화 */
    vPortDefineHeapRegions( xHeapRegions );

    /* 애플리케이션 코드 */
}
```

메모리 할당 방식

□ 예제 메모리 할당 방식

○ Heap_5 초기화 :vPortDefineHeapRegions() API(계속)

- Listing 3.3은 세 RAM 영역을 올바르게 기술하지만, 모든 RAM을 Heap에 할당하여 다른 변수가 사용할 RAM을 남기지 않으므로 실제로 사용할 수 있는 예는 아니다.
- 빌드의 Link 단계에서는 각 변수에 RAM 주소를 할당한다. Linker가 사용할 수 있는 RAM은 일반적으로 Linker Script와 같은 Linker 설정 파일에 기술한다.
- Figure 3.4 B에서는 Linker Script가 RAM1 정보만 포함하고 RAM2와 RAM3 정보는 포함하지 않는다고 가정한다. 따라서 Linker는 변수를 RAM1에 배치하고, RAM1의 0x0001nnnn 주소 이상 영역만 heap_5에서 사용할 수 있도록 남긴다. 0x0001nnnn의 실제 값은 애플리케이션에 포함된 모든 변수의 총 크기에 따라 달라진다. RAM2와 RAM3는 Linker가 사용하지 않았으므로 전체를 heap_5가 사용할 수 있다.
- Listing 3.3처럼 RAM1 전체를 heap_5에 할당하면 0x0001nnnn보다 낮은 Heap 영역이 일반 변수가 사용하는 RAM과 겹칠 수 있다.
- 첫 번째 HeapRegion_t의 시작 주소를 0x00010000 대신 0x0001nnnn으로 설정하면 겹침을 피할 수 있지만 권장하는 방법은 아니다. 그 이유는 다음과 같다.
 - 정확한 시작 주소를 구하기 어려울 수 있다.
 - 향후 빌드에서 Linker가 사용하는 RAM의 양이 바뀌면 HeapRegion_t의 시작 주소도 수정해야 한다.
 - Build Tool은 Linker가 사용하는 RAM과 heap_5가 사용하는 RAM의 겹침을 알지 못하므로 애플리케이션 개발자에게 경고할 수 없다.

메모리 할당 방식

□ 예제 메모리 할당 방식

○ Heap_5 초기화 :vPortDefineHeapRegions() API(계속)

- Listing 3.4는 더 편리하고 유지보수하기 쉬운 방법을 보여준다. ucHeap 배열을 일반 변수로 선언하면 Linker가 이 배열을 RAM1에 배치한다. xHeapRegions 배열의 첫 번째 HeapRegion_t가 ucHeap의 시작 주소와 크기를 기술하므로 ucHeap이 heap_5가 관리하는 메모리의 일부가 된다.
- Listing 3.4 RAM2와 RAM3 전체 및 RAM1의 일부만 기술하는 HeapRegion_t 배열

```
#define RAM2_START_ADDRESS ( ( uint8_t * ) 0x00020000 )
#define RAM2_SIZE          ( 32 * 1024 )
#define RAM3_START_ADDRESS ( ( uint8_t * ) 0x00030000 )
#define RAM3_SIZE          ( 32 * 1024 )

#define RAM1_HEAP_SIZE     ( 30 * 1024 )
static uint8_t ucHeap[ RAM1_HEAP_SIZE ];

const HeapRegion_t xHeapRegions[] =
{
    { ucHeap, RAM1_HEAP_SIZE },
    { RAM2_START_ADDRESS, RAM2_SIZE },
    { RAM3_START_ADDRESS, RAM3_SIZE },
    { NULL, 0 } /* 배열의 끝 */
};
```

메모리 할당 방식

□ 예제 메모리 할당 방식

○ Heap_5 초기화 :vPortDefineHeapRegions() API(계속)

➤ Listing 3.4 방식의 장점은 다음과 같다.

- Hard-Coded Start Address를 사용할 필요가 없다.
- HeapRegion_t에 사용하는 주소를 Linker가 자동으로 설정하므로, 향후 Linker가 사용하는 RAM의 양이 변해도 올바른 주소를 유지한다.
- heap_5가 사용하는 RAM이 Linker가 RAM1에 배치한 다른 데이터와 겹칠 수 없다.
- ucHeap이 너무 크면 애플리케이션이 Link 단계에서 실패하므로 문제를 빌드 시점에 발견할 수 있다.

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ Heap 시작 주소 정의

- heap_1, heap_2, heap_4는 configTOTAL_HEAP_SIZE 크기의 정적 배열에서 메모리를 할당한다. 이 절에서는 이 세 방식을 통칭하여 heap_n이라고 부른다.
- 때로는 Heap을 특정 메모리 주소에 배치해야 한다. 예를 들어 동적으로 생성한 Task의 Stack은 Heap에서 할당하므로, 느린 외부 메모리가 아니라 빠른 내부 메모리에 Heap을 배치해야 할 수 있다.
- configAPPLICATION_ALLOCATED_HEAP 컴파일 타임 설정을 사용하면 heap_n.c가 배열을 선언하는 대신 애플리케이션 코드에서 배열을 직접 선언할 수 있다. 이를 통해 개발자가 Heap 배열의 시작 주소를 지정할 수 있다.
- 원문은 configAPPLICATION_ALLOCATED_HEAP을 1로 설정하거나 정의하지 않은 경우 애플리케이션이 configTOTAL_HEAP_SIZE 크기의 uint8_t 배열 ucHeap을 할당해야 한다고 설명한다.
- 특정 주소에 변수를 배치하는 문법은 컴파일러마다 다르므로 해당 컴파일러 문서를 확인해야 한다. 원문은 GCC와 IAR의 예를 제시한다.

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ Heap 시작 주소 정의(계속)

- Listing 3.5 GCC 문법을 사용하여 heap_4 배열을 .my_heap 메모리 섹션에 배치하는 예

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ]  
    __attribute__ ( ( section( ".my_heap" ) ) );
```

- Listing 3.6 IAR 문법을 사용하여 heap_4 배열을 절대 주소 0x20000000에 배치하는 예

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] @ 0x20000000;
```

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ xPortGetFreeHeapSize() API 함수

- xPortGetFreeHeapSize()는 함수를 호출한 시점에 Heap에 남아 있는 Free Byte 수를 반환한다. Heap Fragmentation에 대한 정보는 제공하지 않는다. heap_3에는 이 함수가 구현되어 있지 않다.

```
size_t xPortGetFreeHeapSize( void );
```

- 반환값은 함수 호출 시점에 Heap에서 아직 할당되지 않은 바이트 수이다.

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ xPortGetMinimumEverFreeHeapSize() API 함수

- xPortGetMinimumEverFreeHeapSize()는 FreeRTOS 애플리케이션이 실행을 시작한 이후 Heap에 존재했던 최소 Free Byte 수를 반환한다.
- 이 값은 애플리케이션이 Heap 고갈에 얼마나 가까이 갔는지를 나타낸다. 예를 들어 반환값이 200이면 실행 이후 어느 시점에 Heap 여유 공간이 200바이트까지 감소했다는 의미이다.
- Heap 크기를 최적화하는 데에도 사용할 수 있다. 가장 Heap 사용량이 큰 코드를 실행한 뒤 반환값이 2000이라면, configTOTAL_HEAP_SIZE를 최대 2000바이트까지 줄일 여지가 있다고 판단할 수 있다.

```
size_t xPortGetMinimumEverFreeHeapSize( void );
```

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ vPortGetHeapStats() API 함수

- heap_4와 heap_5는 vPortGetHeapStats()를 구현한다. 이 함수는 유일한 매개변수로 참조 전달된 HeapStats_t 구조체에 Heap 상태 정보를 채운다.

- Listing 3.9 vPortGetHeapStatus() API 함수 프로토타입(원문 캡션 표기)

```
void vPortGetHeapStats( HeapStats_t *xHeapStats );
```

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ vPortGetHeapStats() API 함수(계속)

- Listing 3.10 HeapStatus_t() 구조체(원문 캡션 표기)

```
typedef struct xHeapStats
{
    /* 현재 사용 가능한 총 Heap 크기: 모든 Free Block의 합 */
    size_t xAvailableHeapSpaceInBytes;

    /* 현재 가장 큰 Free Block의 크기 */
    size_t xSizeOfLargestFreeBlockInBytes;

    /* 현재 가장 작은 Free Block의 크기 */
    size_t xSizeOfSmallestFreeBlockInBytes;

    /* 현재 Free Memory Block의 개수 */
    size_t xNumberOfFreeBlocks;

    /* 부팅 이후 존재했던 최소 총 Free Memory */
    size_t xMinimumEverFreeBytesRemaining;

    /* 유효한 메모리 블록을 반환한 pvPortMalloc() 호출 횟수 */
    size_t xNumberOfSuccessfulAllocations;

    /* 메모리 블록을 성공적으로 해제한 vPortFree() 호출 횟수 */
    size_t xNumberOfSuccessfulFrees;
} HeapStats_t;
```

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ Task별 Heap 사용 통계 수집

- vTaskGetInfo() API는 Task 정보를 TaskStatus_t 구조체에 채운다. FreeRTOSConfig.h에서 configTRACK_TASK_MEMORY_ALLOCATIONS를 1로 설정하면 구조체에 다음 정보가 추가된다.
 - 해당 Task가 pvPortMalloc()을 호출한 횟수
 - 해당 Task가 vPortFree()를 호출한 횟수
 - vTaskGetInfo() 호출 시점에 해당 Task가 할당했지만 어떤 Task에서도 아직 해제하지 않은 Heap Byte 수
 - 해당 Task가 시작된 이후 어느 시점이든 해당 Task가 할당한 Heap Memory의 최대량

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ Malloc Failed Hook 함수

- 표준 malloc()과 마찬가지로 pvPortMalloc()은 요청한 RAM을 할당할 수 없으면 NULL을 반환한다.
- Malloc Failed Hook 또는 Callback은 pvPortMalloc()이 NULL을 반환할 때 호출되는 애플리케이션 제공 함수이다. Callback을 사용하려면 FreeRTOSConfig.h에서 configUSE_MALLOC_FAILED_HOOK을 1로 설정해야 한다.
- 동적 메모리 할당을 사용하여 Kernel Object를 생성하는 FreeRTOS API 내부에서 Malloc Failed Hook이 호출되면 해당 객체는 생성되지 않는다.
- configUSE_MALLOC_FAILED_HOOK이 1이면 애플리케이션은 다음 이름과 프로토타입의 Hook 함수를 제공해야 한다. 함수 내부 동작은 애플리케이션의 요구사항에 맞게 구현한다.

```
void vApplicationMallocFailedHook( void );
```

- 많은 FreeRTOS Demo Application은 메모리 할당 실패를 Fatal Error로 처리한다. 그러나 원문은 Production System에서는 할당 실패에서 정상적으로 복구하는 것이 더 적절한 방식이라고 설명한다.

메모리 할당 함수

□ Heap관련 유틸리티 함수와 매크로

○ Task Stack을 빠른 메모리에 배치

- Stack은 매우 빈번하게 읽고 쓰므로 빠른 메모리에 배치하는 것이 좋다. 그러나 Heap 전체를 반드시 같은 빠른 메모리에 둘 필요는 없을 수 있다.
- FreeRTOS는 `pvPortMallocStack()`과 `vPortFreeStack()` 매크로를 사용하여 FreeRTOS API 내부에서 할당하는 Stack에 별도의 메모리 할당자를 지정할 수 있도록 한다.
- Stack도 `pvPortMalloc()`이 관리하는 Heap에서 가져오려면 `pvPortMallocStack()`과 `vPortFreeStack()`을 정의하지 않는다. 이 경우 각각 `pvPortMalloc()`과 `vPortFree()` 호출로 기본 매핑된다.
- Stack을 별도의 빠른 RAM에서 할당하려면 다음과 같이 애플리케이션 제공 함수로 매크로를 매핑한다.

- Listing 3.12 `pvPortMallocStack()`과 `vPortFreeStack()`을 애플리케이션 정의 메모리 할당 함수로 매핑하는 예

```
void *pvMallocFastMemory( size_t xWantedSize );  
void vPortFreeFastMemory( void *pvBlockToFree );  
  
#define pvPortMallocStack( x ) pvMallocFastMemory( x )  
#define vPortFreeStack( x ) vPortFreeFastMemory( x )
```

메모리 할당

□ 정적 메모리 할당 사용

- 동적 메모리 할당의 단점을 피하기 위해 정적 메모리 할당을 사용할 수 있다. 정적 할당에서는 애플리케이션에 필요한 각 메모리 블록을 개발자가 명시적으로 생성한다.
- 정적 메모리 할당의 주요 장점은 다음과 같다.
 - 필요한 모든 메모리의 크기를 컴파일 시점에 알 수 있다.
 - 모든 메모리 할당 동작이 결정적이다.
- 이러한 장점과 함께 몇 가지 복잡성도 생긴다. 대표적으로 일부 Kernel Memory를 관리하기 위한 사용자 함수가 추가로 필요하고, 모든 정적 메모리가 적절한 Scope에 선언되도록 해야 한다.

메모리 할당

○ 정적 메모리 할당 활성화

- FreeRTOSConfig.h에서 configSUPPORT_STATIC_ALLOCATION을 1로 설정하면 정적 메모리 할당을 활성화한다. 이 설정을 사용하면 Kernel의 Static 버전 생성 함수들이 활성화된다.

함수	조건/설명
xTaskCreateStatic	정적 메모리를 사용하여 Task 생성
xEventGroupCreateStatic	정적 메모리를 사용하여 Event Group 생성
xEventGroupGetStaticBuffer	Event Group의 Static Buffer 접근
xQueueGenericCreateStatic	정적 메모리를 사용하여 Queue 생성
xQueueGenericGetStaticBuffers	Queue의 Static Buffer 접근
xQueueCreateMutexStatic	configUSE_MUTEXES가 1인 경우
xQueueCreateCountingSemaphoreStatic	configUSE_COUNTING_SEMAPHORES가 1인 경우
xStreamBufferGenericCreateStatic	정적 메모리를 사용하여 Stream Buffer 생성
xStreamBufferGetStaticBuffers	Stream Buffer의 Static Buffer 접근
xTimerCreateStatic	configUSE_TIMERS가 1인 경우
xTimerGetStaticBuffer	configUSE_TIMERS가 1인 경우

- 각 함수의 자세한 사용법은 해당 기능을 설명하는 장에서 다룬다.

메모리 할당

□ 정적 메모리 할당 사용

○ 정적 내부 Kernel Memory

- 정적 메모리 할당을 활성화하면 Idle Task와 Timer Task(활성화한 경우)는 사용자 함수가 제공하는 정적 메모리를 사용한다.

사용자 함수	사용 조건
vApplicationGetTimerTaskMemory	configUSE_TIMERS가 1인 경우 Timer Task 메모리 제공
vApplicationGetIdleTaskMemory	Main Idle Task 메모리 제공

메모리 할당

□ 정적 메모리 할당 사용

○ vApplicationGetTimerTaskMemory

- configSUPPORT_STATIC_ALLOCATION과 configUSE_TIMERS를 모두 활성화하면 Kernel은 vApplicationGetTimerTaskMemory()를 호출한다.
- 이 함수는 Timer Task의 TCB용 메모리 버퍼와 Timer Task Stack용 버퍼를 생성하여 Kernel에 반환하고, Timer Task Stack의 크기도 반환한다.
- Listing 3.13 vApplicationGetTimerTaskMemory의 일반적인 구현

```
void vApplicationGetTimerTaskMemory(
    StaticTask_t **ppxTimerTaskTCBBuffer,
    StackType_t **ppxTimerTaskStackBuffer,
    uint32_t *pulTimerTaskStackSize )
{
    /* 함수 종료 후에도 유지되어야 하므로 static으로 선언 */
    static StaticTask_t xTimerTaskTCB;
    static StackType_t uxTimerTaskStack[ configMINIMAL_STACK_SIZE ];

    *ppxTimerTaskTCBBuffer = &xTimerTaskTCB;
    *ppxTimerTaskStackBuffer = uxTimerTaskStack;
    *pulTimerTaskStackSize =
        sizeof( uxTimerTaskStack ) / sizeof( *uxTimerTaskStack );
}
```

- SMP를 포함한 시스템에서도 Timer Task는 하나만 존재하므로, vApplicationGetTimerTaskMemory() 내부에 Static Buffer를 선언하고 그 포인터를 Kernel에 반환하는 방식이 유효한 해결 방법이 된다.

메모리 할당

□ 정적 메모리 할당 사용

○ vApplicationGetIdleTaskMemory

- Core에 실행할 예약 작업이 없을 때 Idle Task가 실행된다. Idle Task는 일부 Housekeeping 작업을 수행하고, 활성화되어 있다면 사용자가 정의한 vTaskIdleHook()도 호출할 수 있다.
- 대칭형 멀티프로세싱(SMP) 시스템에서는 나머지 Core마다 Housekeeping을 수행하지 않는 추가 Idle Task가 존재하며, 이 Task들은 내부적으로 configMINIMUM_STACK_SIZE 바이트의 정적 메모리를 할당받는다.
- vApplicationGetIdleTaskMemory()는 애플리케이션이 “Main” Idle Task에 필요한 Buffer를 생성하여 제공할 수 있도록 호출된다.
- Listing 3.14 vApplicationGetIdleTaskMemory의 일반적인 구현

```
void vApplicationGetIdleTaskMemory(
    StaticTask_t **ppxIdleTaskTCBBuffer,
    StackType_t **ppxIdleTaskStackBuffer,
    uint32_t *pulIdleTaskStackSize )
{
    static StaticTask_t xIdleTaskTCB;
    static StackType_t uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];

    *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
    *ppxIdleTaskStackBuffer = uxIdleTaskStack;
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
```

메모리 할당

□ 정적 메모리 할당 사용

○ vApplicationGetIdleTaskMemory

원문 표기 참고

Listing 3.14를 설명하는 문장에는 vApplicationIdleTaskMemory()라는 이름이 등장하지만, 실제 코드와 절 제목에는 vApplicationGetIdleTaskMemory()가 사용된다.

강의용 핵심 정리

- FreeRTOS Kernel Object의 메모리는 정적 또는 동적으로 할당할 수 있다.
- FreeRTOS의 동적 메모리 할당은 pvPortMalloc()과 vPortFree()를 사용하며, heap_1부터 heap_5까지 다섯 가지 예제 구현을 제공한다.
- heap_1은 해제를 지원하지 않지만 결정적이고 Fragmentation이 발생하지 않는다.
- heap_2는 Best-Fit을 사용하지만 인접 Free Block을 병합하지 않아 새 설계에는 권장하지 않는다.
- heap_4는 First-Fit과 Free Block 병합을 사용하여 일반적인 동적 할당에 적합하다.
- heap_5는 heap_4의 알고리즘을 사용하면서 여러 비연속 RAM 영역을 하나의 Heap으로 구성할 수 있다.
- xPortGetFreeHeapSize(), xPortGetMinimumEverFreeHeapSize(), vPortGetHeapStats()를 사용하여 Heap 상태를 확인할 수 있다.
- 정적 메모리 할당을 사용하면 필요한 메모리를 컴파일 시점에 결정할 수 있으며, Idle Task와 Timer Task용 메모리도 애플리케이션이 제공한다.



Task 관리



Introduction

□ 본 장의 구성

- FreeRTOS가 각 Task에 처리 시간을 할당하는 방법
 - 특정 시점에 실행할 Task를 선택하는 기준
 - Task의 상대 Priority가 시스템 동작에 미치는 영향
- Task 함수와 하나 이상의 Task Instance 생성
- Task 상태와 상태 전이
- 주기적 처리, Idle Task, Priority 변경과 Task 삭제
- Thread Local Storage와 Scheduling Algorithm



Task 관리

□ 기본 형식

○ FreeRTOS Task는 C 함수로 구현한다.

- 하나의 void 포인터 매개변수를 받고 void를 반환한다.
- 일반적으로 무한 루프 안에서 계속 실행한다.

```
void vATaskFunction( void * pvParameters )
```

□ 중요한 규칙

○ Task 함수는 어떤 경로로도 return해서는 안 된다.

- Task가 더 이상 필요하지 않으면 vTaskDelete()를 호출한다.

○ 하나의 Task 함수 정의로 여러 Task Instance를 만들 수 있다.

- 자동 변수는 각 Instance의 Stack에 별도로 존재한다.
- static 변수는 모든 Instance가 같은 객체를 공유한다.



Task 관리

□ 일반적인 Task 함수의 구조

- 자동 변수와 static 변수의 저장 위치를 구분해야 한다.

```
void vATaskFunction( void * pvParameters )
{
    long lStackVariable = 0;
    static long lStaticVariable = 0;

    for( ;; )
    {
        /* Task 기능을 구현하는 코드 */
    }

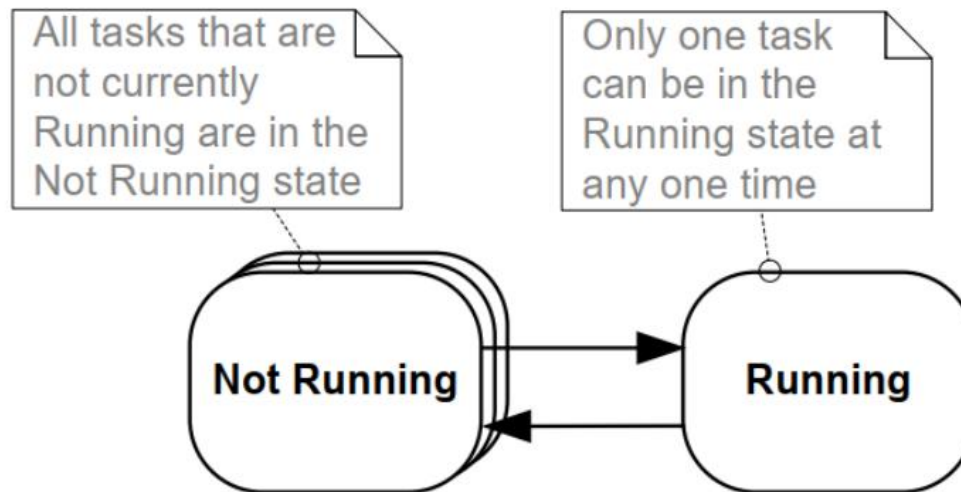
    /* 무한 루프를 빠져나왔다면 Task를 삭제한다. */
    vTaskDelete( NULL );
}
```



Task 관리

□ Running과 Not Running

- Running Task는 Processor가 실제로 코드를 실행하는 Task이다.
- Not Running Task는 실행 상태를 저장하고 Scheduler의 선택을 기다린다.
- Scheduler만 Task를 Running 상태로 넣거나 뺄 수 있다.
- Running으로 전환되면 이전에 멈춘 지점부터 실행을 계속한다.



Figure

4.1 Top level task states and transitions

- Running과 Not Running의 두 상태에서 시작한다. 이후 Not Running을 Ready, Blocked, Suspended로 세분화한다.
- Not Running 상태의 Task가 Running 상태로 전환되는 것을 switched in 또는 swapped in 이라고 하며 반대 방향은 switched out 또는 swapped out이라고 한다.



Task 관리

□ Task 생성 API

- Static, Restricted, Affinity는 메모리 할당, 권한, 실행 Core 지정 방법의 차이를 나타낸다.

API	특징
xTaskCreate()	가장 단순한 동적 메모리 기반 생성
xTaskCreateStatic()	TCB와 Stack 메모리를 호출자가 제공
xTaskCreateRestricted()	제한된 권한으로 실행되는 Task 생성
xTaskCreateRestrictedStatic()	Restricted Task를 정적 메모리 방식으로 생성
xTaskCreateAffinitySet()	SMP 환경에서 실행 Core 지정
xTaskCreateStaticAffinitySet()	Static Allocation과 Core Affinity를 함께 사용

- 각 Task는 TCB와 Task Stack을 위한 두 개의 RAM 블록을 필요로 한다.



Task 관리

☐ Dynamic Allocation

- 이름에 Static이 없는 API는 실행 중에 System Heap에서 RAM을 할당한다.
- Task Control Block(TCB)와 Task Stack용 Memory를 확보한다.

☐ Static Allocation

- 이름에 Static이 있는 API는 호출자가 미리 할당한 Memory를 전달한다.
- 실행 중 Memory Allocation을 피해야 하는 시스템에 적합하다.

☐ Restricted와 Affinity

- Restricted API는 Memory 접근이 제한된 Task를 만들 수 있는 port에서 사용한다.
 - Restricted가 없는 API로 생성한 Task는 일반적으로 Privileged Mode 로 실행하며 시스템 전체 Memory Map에 접근할 수 있다.
- Affinity API는 SMP를 지원하는 FreeRTOS Port에서 해당 API를 사용하여 Task가 실행될 Core를 지정할 수 있다.



Task 관리

□ 가장 단순한 동적 메모리 기반 Task 생성 함수

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode,  
    const char * const pcName,  
    configSTACK_DEPTH_TYPE usStackDepth,  
    void * pvParameters,  
    UBaseType_t uxPriority,  
    TaskHandle_t * pxCreatedTask  
);
```

○ 반환값

- pdPASS : Task를 성공적으로 생성했다.
- pdFAIL : Task를 생성할 Heap Memory가 부족하다.



Task 관리

□ Parameter의 의미

Parameter	설명
pvTaskCode	Task를 구현하는 함수의 포인터
pcName	디버깅에 사용하는 사람이 읽을 수 있는 Task 이름
usStackDepth	Task Stack의 깊이. byte가 아니라 word 단위
pvParameters	Task 함수의 void * 매개변수로 전달할 값
uxPriority	초기 Priority. 최저값은 0, 최고값은 configMAX_PRIORITIES-1
pxCreatedTask	생성된 Task Handle을 저장할 위치. 필요 없으면 NULL

○ Stack 용량은 Stack Depth × Stack Word 폭으로 계산한다.



Task 관리

☐ pcName

- 사람이 읽을 수 있는 디버깅용 Task 이름이다.
- FreeRTOS Kernel 자체는 동작에는 사용하지 않으며 최대 길이는 configMAX_TASK_NAME_LEN이 정한다.

☐ usStackDepth

- Stack 크기를 byte가 아닌 word 단위로 지정한다.
- Stack word가 32 bit이고 값이 128이면 512 byte를 할당한다.

☐ configSTACK_DEPTH_TYPE

- 정의하지 않으면 기본 자료형은 uint16_t이다.
- Stack 용량이 65,535를 넘을 수 있으면 unsigned long이나 size_t와 같은 더 큰 자료형을 사용한다.



Task 관리

□ 동일한 Priority를 가진 두 Task

- 두 Task는 출력 문자열만 다르고 나머지 구조는 동일하다.

```
void vTask1( void * pvParameters )
{
    volatile unsigned long ulCount;
    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        for( ulCount = 0;
            ulCount < mainDELAY_LOOP_COUNT;
            ulCount++ ) {
            /*단순한 Busy-Wait 지연*/
        }
    }
}
```

```
void vTask2( void * pvParameters )
{
    volatile unsigned long ulCount;
    for( ;; )
    {
        vPrintLine( "Task 2 is running" );
        for( ulCount = 0;
            ulCount < mainDELAY_LOOP_COUNT;
            ulCount++ ) {
            /*단순한 Busy-Wait 지연*/
        }
    }
}
```

- 주기적 출력 효과를 만들기 위해 Null Loop를 임시 지연 방법으로 사용한다.



Task 관리

□ Scheduler 시작 절차

- Task 생성 후 vTaskStartScheduler()를 호출한다.

```
int main( void )
{
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    vTaskStartScheduler();

    /* 정상이라면 여기에 도달하지 않는다. */
    for( ;; );
}
```

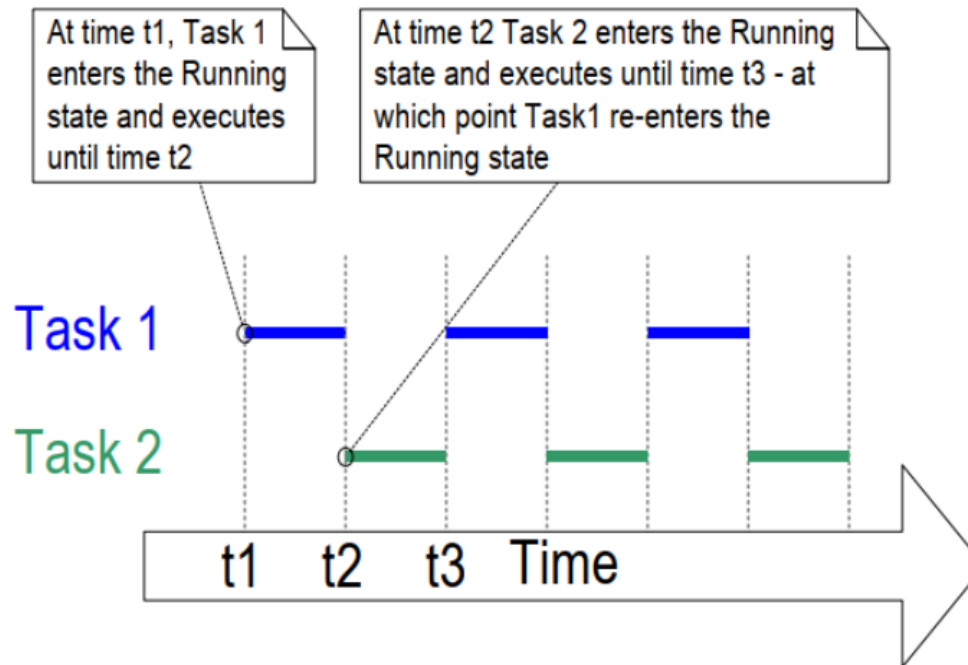
- main()의 Stack에 선언한 지역 변수는 scheduler 시작 후 Task에서 참조하지 않는다.
- Scheduler가 시작된 뒤 다른 Task 내부에서도 새로운 Task를 생성할 수 있다.



Task 관리

□ 동일 Priority Task의 실행

- 단일 Core에서 동시에 실행하는 것이 아니라 Running 상태를 빠르게 번갈아 사용한다.



Figure

4.3 The actual execution pattern of the two Example 4.1 tasks

- 한 Task가 Running으로 전환되면 다른 Task는 Not Running으로 전환된다.



Task 관리

☐ 단일 Core의 실행

- 두 Task가 동시에 실행되는 것이 아니라 Scheduler의 선택에 따라 번갈아 Running 상태가 된다.
- 한 Task가 Running 상태가 되면 다른 Task는 Not Running 상태로 전환된다.

☐ Windows Simulator의 출력

- 원문의 Simulator에서는 두 Task가 한 번씩 번갈아 출력하는 모습이 뚜렷하다.
- 실제 Embedded Target에서는 출력 함수의 속도에 따라 한 Task가 여러 번 출력한 뒤 교체될 수 있다.

☐ main() Stack 주의

- Scheduler 시작 이후 Task가 main()의 지역 변수를 참조하도록 작성하지 않는다.



Task 관리

☐ Scheduler 시작 후 Task 생성

- Task는 main()에서만 생성할 필요가 없다.
- Scheduler가 실행 중이면 다른 Task 내부에서도 생성할 수 있다.

```
void vTask1( void * pvParameters )
{
    volatile unsigned long ul;

    /* Scheduler가 이미 실행 중인 상태에서 Task 2를 생성한다. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ ) { }
    }
}
```



Task 관리

□ 하나의 Task 함수에서 여러 Instance 생성

○ pvParameters로 서로 다른 문자열을 전달하여 중복 코드를 제거한다.

```
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName;
    volatile unsigned long ul;

    pcTaskName = ( char * ) pvParameters;

    for( ;; )
    {
        vPrintLine( pcTaskName );
        for( ul = 0; ul <
mainDELAY_LOOP_COUNT; ul++ ){}
    }
}
```

```
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main( void )
{
    xTaskCreate( vTaskFunction, "Task 1", 1000,
                ( void * ) pcTextForTask1, 1, NULL );

    xTaskCreate( vTaskFunction, "Task 2", 1000,
                ( void * ) pcTextForTask2, 1, NULL );

    vTaskStartScheduler();
    for(;;){}
}
```

➤ 함수 코드는 공유하지만 각 Task는 독립적인 Stack과 자동 변수를 가진다.



Task 관리

☐ Priority 기본 원칙

- Scheduler는 실행 가능한 Task 중 Priority가 가장 높은 Task를 Running 상태로 선택한다.
- 같은 Priority의 Task가 여러 개이면 실행 기회를 번갈아 가진다.
- 0이 최저이고 configMAX_PRIORITIES-1이 최고이다.

☐ Generic Scheduler

- C로 구현되어 모든 FreeRTOS Architecture Port에서 사용할 수 있다.
- Priority 수가 늘면 RAM 사용량과 Worst-Case Execution Time이 증가할 수 있다.
- configUSE_PORT_OPTIMISED_TASK_SELECTION=0으로 선택한다.

☐ Architecture-Optimized Scheduler

- Architecture-specific Assembly를 사용한다.
- 지원 Architecture에서는 Priority 수와 무관하게 Task 선택 시간이 일정하다.
- 32-bit에서는 최대 32, 64-bit에서는 최대 64 Priority Level을 사용한다.
- 지원 Port는 configUSE_PORT_OPTIMISED_TASK_SELECTION=1을 사용한다.



Task 관리

□ configMAX_PRIORITIES와 Task 선택 시간

구분	Generic	Architecture-Optimized
구현	C	Architecture-specific Assembly
Priority 수	고정 상한 없음	32-bit는 32, 64-bit는 64
Worst-Case 시간	Priority 수가 늘면 길어질 수 있음	Priority 수와 관계없이 일정
설정	configUSE_PORT_OPTIMISED_TASK_SELECTION=0	지원 Port에서 configUSE_PORT_OPTIMISED_TASK_SELECTION=1



Task 관리

☐ Architecture-Optimized 구현

- 지원 Port에서는 configUSE_PORT_OPTIMISED_TASK_SELECTION=1을 사용한다.
- 지원 Port는 설정 상수를 정의하지 않아도 최적화 구현을 기본값으로 사용할 수 있다.
- Priority 수와 관계없이 Task 선택 시간이 일정하다.

☐ Generic 구현

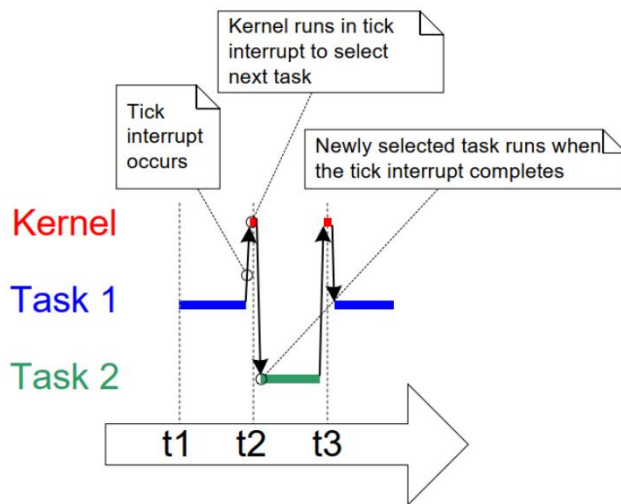
- configUSE_PORT_OPTIMISED_TASK_SELECTION=0으로 설정한다.
- Priority Level 수를 실용적인 범위로 유지하면 RAM 사용량과 Worst-Case 선택 시간을 줄일 수 있다.
- 이식성이 중요하거나 지원되는 최적화 구현이 없을 때 사용한다.



Task 관리

□ Tick 기반 시간 관리

- Scheduler는 주기적인 Tick Interrupt를 사용하여 다음 Task를 선택한다.
- configTICK_RATE_HZ가 Tick Frequency를 정한다.



○ 설정 예

- configTICK_RATE_HZ = 100 Hz
- Tick Period = 10 ms

○ 시간 변환

- pdMS_TO_TICKS()를 사용하여 ms를 Tick 수로 변환한다.

```
TickType_t xTimeInTicks = pdMS_TO_TICKS( 200 );
```

- Task가 Blocked로 들어가거나 더 높은 Priority Task가 Ready가 되면 Tick 사이에도 Task를 교체할 수 있다.



Task 관리

☐ Time Slice와 Tick Period

- Time Slicing을 사용하면 한 Time Slice는 일반적으로 한 Tick Period와 같다.
- `configTICK_RATE_HZ=1000`이면 Tick Period와 Time Slice는 10 ms이다.

☐ `pdMS_TO_TICKS()`

- 시간을 ms 단위로 표현하면 Tick Frequency가 바뀌어도 응용 코드의 시간 설정을 유지하기 쉽다.
- 원문은 `configTICK_RATE_HZ`가 1000보다 큰 경우 `pdMS_TO_TICKS()`를 사용할 수 없다고 설명한다.

☐ Tick Count

- Scheduler 시작 후 발생한 Tick Interrupt의 누적 횟수이다.
- Overflow가 발생해도 일반적인 지연 처리의 일관성은 Kernel이 관리한다.



Task 관리

□ 서로 다른 Priority로 두 Task 생성

- Task 2는 Priority 2, Task 1은 Priority 1로 생성한다.

```
xTaskCreate( vTaskFunction, "Task 1", 1000, ( void * ) pcTextForTask1, 1, NULL );
```

```
xTaskCreate( vTaskFunction, "Task 2", 1000, ( void * ) pcTextForTask2, 2, NULL );
```

○ 실행 결과

- Task 2는 더 높은 Priority이고 항상 실행 가능하므로 계속 Running 상태가 된다.
- Task 1은 CPU 시간을 받지 못해 Starvation 상태에 빠진다.

○ 핵심 원인

- Null Loop 와 출력만 반복하면서 Blocked 상태에 들어가지 않기 때문이다.
- 높은 Priority의 연속 처리 Task 가 낮은 Priority Task 를 완전히 굶길 수 있다.



Task 관리

☐ Event-Driven Task

- 처리할 Event가 없을 때 Blocked 상태로 들어가 CPU를 사용하지 않는다.

상태 또는 Event	설명	대표 예
Temporal Event	지연 시간 만료 또는 절대 시간 도달처럼 시간에 의해 발생	10ms 가 지날 때까지 대기
Synchronization Event	다른 Task 또는 Interrupt에서 발생	Queue 데이터 도착
Timeout	두 조건 중 먼저 만족	데이터 도착 또는 대기 시간 만료

☐ Not Running의 하위 상태

- Ready : 실행할 준비가 되었지만 현재 CPU를 사용하지 않는다.
- Blocked : 시간 또는 동기화 Event를 기다린다.
- Suspended
 - vTaskSuspend()로 진입한다.
 - vTaskResume() 또는 xTaskResumeFromISR()로 해제한다.



Task 관리

☐ Synchronization Event

- Queue, Binary Semaphore, Counting Semaphore, Mutex, Recursive Mutex를 통해 전달할 수 있다.
- Event Group, Stream Buffer, Message Buffer, Direct-to-Task Notification도 Synchronization Event를 만들 수 있다.

☐ Timeout

- Synchronization Event와 최대 대기 시간을 함께 지정할 수 있다.
- Queue Data를 10 ms 대기하면 Data 도착과 10 ms 경과 중 먼저 만족된 조건으로 Blocked 상태를 빠져나온다.

☐ Suspended 상태와 Ready 상태

- Suspended Task는 Resume API가 호출될 때까지 Scheduler가 선택할 수 없다.
- Ready Task는 실행 준비가 되었지만 현재 CPU를 사용하지 않는다.



Task 관리

☐ 상태 정의

- Suspended도 Not Running의 하위 상태이다.
- Suspended 상태의 Task는 Scheduler가 선택할 수 없다.

☐ 상태 전환

- vTaskSuspend()를 호출하면 Suspended 상태에 들어간다.
- vTaskResume() 또는 xTaskResumeFromISR()로만 빠져나올 수 있다.

☐ 사용 빈도

- 대부분의 애플리케이션은 Suspended 상태를 사용하지 않는다.



Task 관리

☐ 상태 정의

- Blocked 상태도 아니고 Suspended 상태도 아닌 Not Running Task이다.
- 실행할 수 있고 실행할 준비가 되었지만 현재 CPU를 사용하지 않는다.

☐ Scheduler의 선택

- Ready Task 중 Priority가 가장 높은 Task를 Running 상태로 전환한다.
- 같은 Priority의 Ready Task가 여러 개이면 설정된 Scheduling Algorithm에 따라 실행 기회를 나눈다.



Task 관리

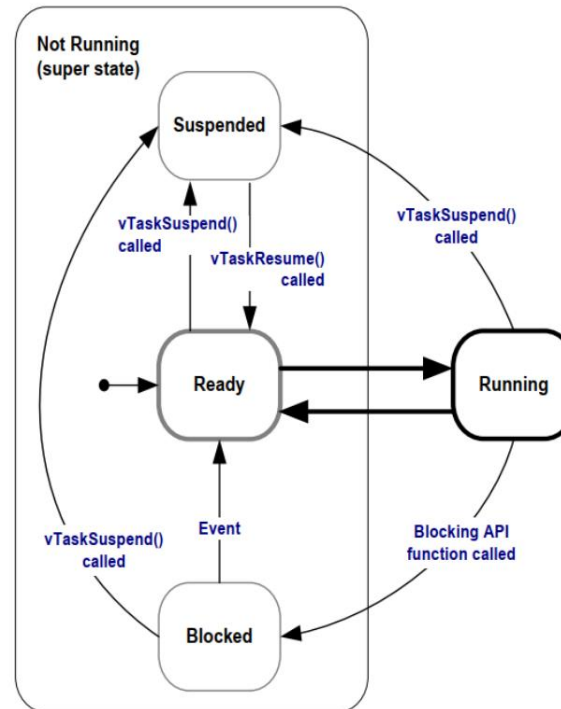
□ 상태와 전이 API

현재 상태	전이 조건	다음 상태
Ready	Scheduler가 선택	Running
Running	Scheduler가 다른 Task를 선택	Ready
Running	시간 또는 동기화 Event 대기	Blocked
Blocked	Event 발생 또는 Timeout 만료	Ready
Running 또는 Ready	vTaskSuspend() 호출	Suspended
Suspended	vTaskResume() 계열 호출	Ready



Task 관리

□ Ready, Running, Blocked, Suspended 상태



task state machine

Figure 4.7 Full

- Ready → Running : Scheduler가 실행할 Task를 선택한다.
- Running → Blocked : Blocking API를 호출한다.
- Blocked → Ready : 기다리던 Event가 발생한다.



Task 관리

□ vTaskDelay() API 함수

- 호출 시점부터 지정한 Tick 수 동안 호출 Task를 Blocked 상태로 둔다.

```
void vTaskDelay( TickType_t xTicksToDelay );
```

```
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName = ( char * ) pvParameters;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

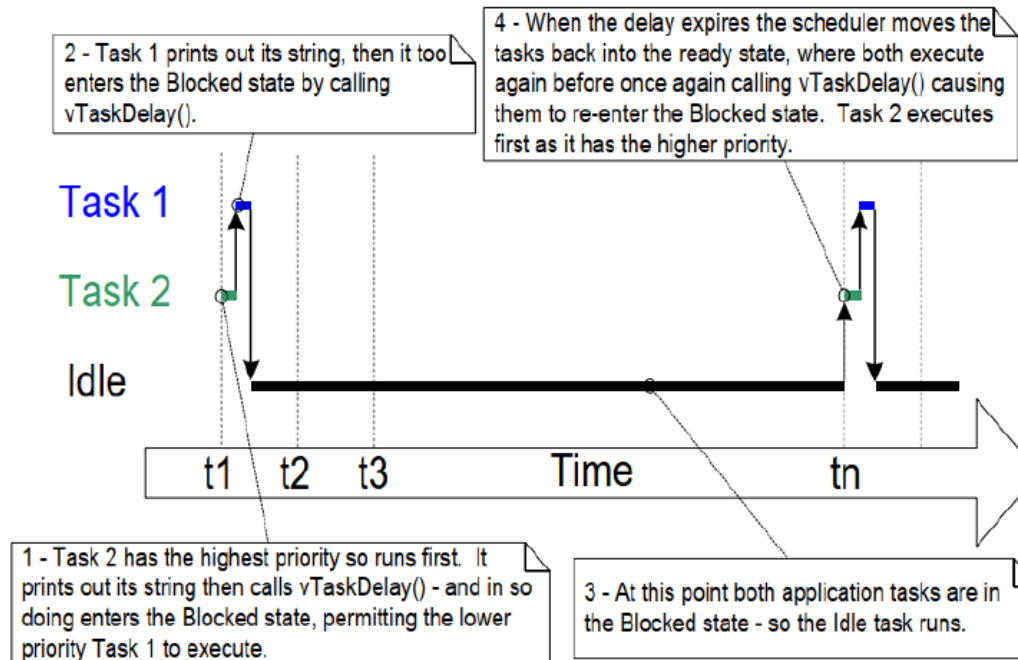
    for( ;; )
    {
        vPrintLine( pcTaskName );
        vTaskDelay( xDelay250ms );
    }
}
```

- INCLUDE_vTaskDelay=1이어야 사용할 수 있다. Tick Count 10,000에서 vTaskDelay(100)을 호출하면 10,100이 될 때까지 Blocked 상태에 머문다.
- xTicksToDelay는 호출 Task가 Ready 상태로 돌아오기 전까지 Blocked 상태에 머무를 Tick 수를 지정한다. pdMS_TO_TICKS()를 사용하면 밀리초를 Tick 수로 변환할 수 있다.
- vTaskDelay(pdMS_TO_TICKS(100))은 약 100ms 동안 Task를 Blocked 상태로 둔다.



Task 관리

- Null Loop를 `vTaskDelay()`로 교체
 - 지연 중에는 CPU 시간을 사용하지 않는다.



- 높은 Priority Task가 Blocked 상태에 있는 동안 낮은 Priority Task가 Running 상태가 될 수 있다.
- 두 Task가 모두 Blocked 상태이면 Idle Task가 실행된다.
- `vTaskDelay()` 사용 시 Task는 실제 작업을 수행할 때만 CPU를 사용하며, Idle Task의 실행 시간은 시스템의 여유 처리 능력을 나타낸다.



Task 관리

□ 고정 주기의 주기적 Task

- `vTaskDelay()`의 지연 종료 시점은 함수를 호출한 시점을 기준으로 한 상대 시간이다.
 - 매 주기마다 정확히 같은 주기로 실행해야 하면 호출 시점의 흔들림이 누적될 수 있다.
- `vTaskDelayUntil()`은 Ready 상태로 전환할 정확한 Tick Count를 계산하므로 고정 주기 Task에 적합하다.

□ 매개변수

- `pxPreviousWakeTime`
 - Task가 직전에 Blocked 상태를 빠져나온 시각을 보관한다.
 - 첫 사용 전 현재 Tick Count로 초기화하며, 이후에는 함수가 자동으로 갱신한다.
- `xTimeIncrement`
 - 주기 길이를 Tick 단위로 지정한다.
 - `pdMS_TO_TICKS()`로 밀리초를 Tick으로 변환할 수 있다.



Task 관리

- 기준 시각을 누적해서 관리하므로 일정한 실행 주기를 유지할 수 있다.

```
void vTaskDelayUntil(  
    TickType_t * pxPreviousWakeTime,  
    TickType_t xTimeIncrement  
);
```

```
void vTaskFunction( void * pvParameters )  
{  
    char * pcTaskName = ( char * ) pvParameters;  
    TickType_t xLastWakeTime;  
    xLastWakeTime = xTaskGetTickCount();  
    for( ;; )  
    {  
        vPrintLine( pcTaskName );  
        vTaskDelayUntil( &xLastWakeTime, pdMS_TO_TICKS( 250 ) );  
    }  
}
```



Task 관리

- 두 개의 Continuous Processing Task와 하나의 Periodic Task를 함께 실행
 - Priority 1의 Continuous Processing Task는 Blocked API를 호출하지 않아 항상 Ready 또는 Running 상태이다.

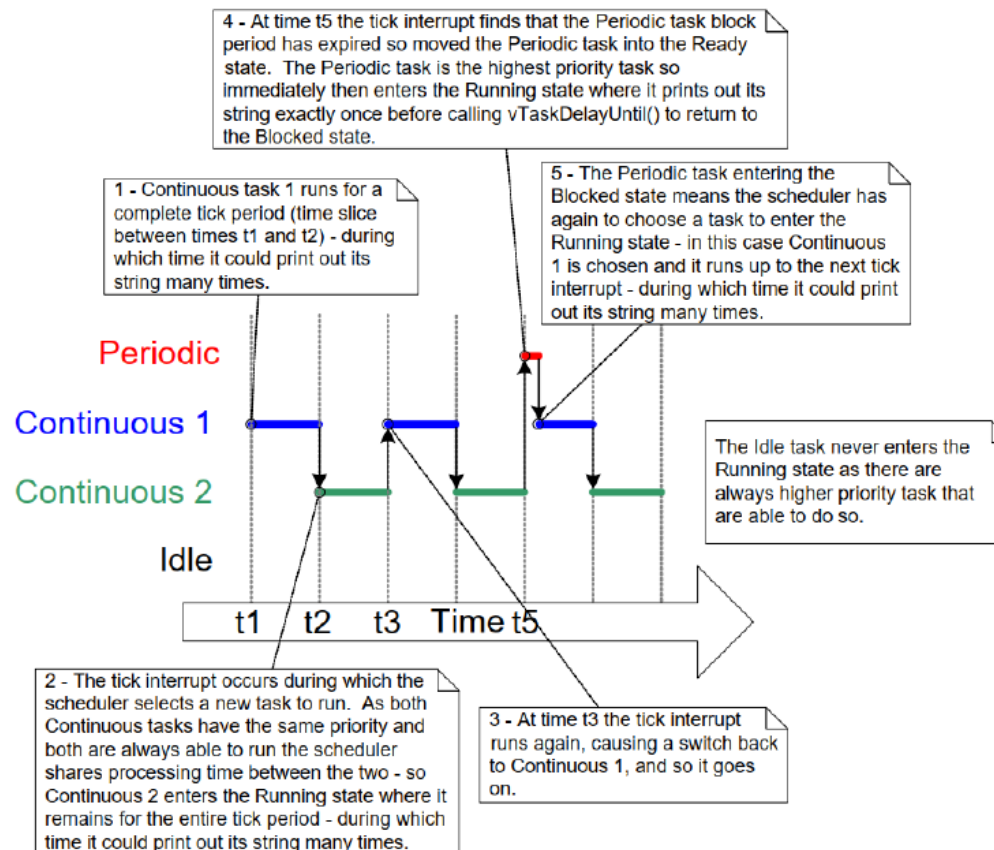


Figure 4.12 The execution pattern of Example 4.6



Task 관리

□ Periodic Task는 출력 후 vTaskDelayUntil()을 호출하여 다음 주기까지, Blocked 상태로 들어간다.

```
void vContinuousProcessingTask( void *  
pvParameters )  
{  
    char * pcTaskName = ( char * )  
pvParameters;  
    for( ;; )  
    {  
        vPrintLine( pcTaskName );  
    }  
}
```

```
void vPeriodicTask( void * pvParameters )  
{  
    TickType_t xLastWakeTime =  
xTaskGetTickCount();  
    const TickType_t xDelay3ms = pdMS_TO_TICKS(  
3 );  
    for( ;; )  
    {  
        vPrintLine( "Periodic task is running" );  
        vTaskDelayUntil( &xLastWakeTime, xDelay3ms  
);  
    }  
}
```

□ 높은 Priority의 Periodic Task가 Ready 상태가 되는 순간 낮은 Priority의 Continuous Task를 Preempt한다.

- Periodic Task가 다시 Blocked 상태로 들어가면
- 두 Continuous Task가 동일 Priority에서 CPU를 공유한다.



Task 관리

□ Scheduler는 vTaskStartScheduler()가 호출될 때 Idle Task를 자동으로 생성한다.

- 시스템에는 항상 Running 상태에 들어갈 수 있는 Task가 하나 이상 있어야 한다.
- Idle Task는 거의 단순 Loop만 수행하므로 항상 실행 가능하다.
- Priority는 가능한 최저값인 0이다.
- 더 높은 Priority Task가 Ready 상태가 되면 즉시 Running 상태를 내어준다.

□ configIDLE_SHOULD_YIELD

- Priority 0의 Application Task와 CPU 시간을 공유하는 방식을 조절한다.

□ Task 삭제와 Idle Task

- 자기 자신을 삭제한 Task의 Kernel Resource는 Idle Task가 정리한다.
- 따라서 이런 시스템에서는 Idle Task를 Starvation시켜서는 안 된다.



Task 관리

□ Idle Hook 또는 Idle Callback은 Idle Task의 Loop가 한 번 반복될 때마다 자동으로 호출되는 Application 함수이다.

○ Idle 시간에 낮은 Priority의 기능을 수행하도록 활용할 수 있다.

```
void vApplicationIdleHook( void );
```

○ 대표적인 활용 방법

- 별도의 Application Task 없이 Background 또는 Continuous Processing 기능을 실행한다.
- Idle Task가 실행된 시간을 측정하여 시스템의 남은 처리 능력을 추정한다.
- Application 작업이 없을 때 Processor를 Low-Power Mode로 전환한다.



Task 관리

□ 4.8.2 Idle Task Hook 구현 시 제한 사항

- Idle Hook은 자신을 Block 또는 Suspend해서는 안 된다.
- 자기 삭제 Task를 사용하면 적절한 시간 안에 반환해야 한다.

```
volatile unsigned long ullIdleCycleCount = 0UL;  
  
void vApplicationIdleHook( void )  
{  
    ullIdleCycleCount++;  
}
```

- 예제 4.7은 Idle Hook에서 전역 Counter를 증가시킨다.
- Counter의 증가량을 통해 Idle 시간이 얼마나 많은지 관찰할 수 있다.



Task 관리

□ Idle Hook이 실제로 호출되게 하려면 configUSE_IDLE_HOOK을 1로 설정한다.

○ 주기 Task에서 ulIdleCycleCount를 함께 출력한다.

```
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName = ( char * ) pvParameters;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    for( ;; )
    {
        vPrintLineAndNumber( pcTaskName, ulIdleCycleCount );
        vTaskDelay( xDelay250ms );
    }
}
```

□ 원문의 Windows 예제에서는 Application Task가 한 번 실행하는 사이 Idle Hook이 약 수백만 번 실행된다.

○ 정확한 횟수는 Hardware 성능에 따라 달라진다.



Task 관리

□ vTaskPrioritySet() API 함수와 uxTaskPriorityGet() API 함수

```
void vTaskPrioritySet(  
    TaskHandle_t xTask,  
    UBaseType_t uxNewPriority  
);  
  
UBaseType_t uxTaskPriorityGet(  
    TaskHandle_t xTask  
);
```

- vTaskPrioritySet()은 Scheduler가 시작된 이후 Task의 Priority를 변경한다.
- xTask에 NULL을 전달하면 호출 Task 자신의 Priority를 변경한다.
- uxNewPriority가 configMAX_PRIORITIES-1보다 크면 최대값으로 제한한다.
- INCLUDE_vTaskPrioritySet을 1로 설정해야 사용할 수 있다.
- uxTaskPriorityGet()은 특정 Task의 현재 Priority를 반환한다.
- INCLUDE_uxTaskPriorityGet을 1로 설정해야 사용할 수 있으며, NULL은 호출 Task 자신을 뜻한다.



Task 관리

□ Task 1과 Task 2 구현

```
TaskHandle_t xTask2Handle = NULL;

void vTask1( void * pvParameters )
{
    UBaseType_t uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        vPrintLine( "About to raise the Task 2 priority" );
        vTaskPrioritySet( xTask2Handle, uxPriority + 1 );
    }
}
```

```
void vTask2( void * pvParameters )
{
    UBaseType_t uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        vPrintLine( "Task 2 is running" );
        vPrintLine( "About to lower the Task 2 priority" );
        vTaskPrioritySet( NULL, uxPriority - 2 );
    }
}
```

- Scheduler는 항상 가장 높은 Ready Task를 즉시 Running 상태로 선택한다.



Task 관리

- Task 2 Handle을 생성 시 저장한다.

```
TaskHandle_t xTask2Handle = NULL;

int main( void )
{
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );

    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, &xTask2Handle );

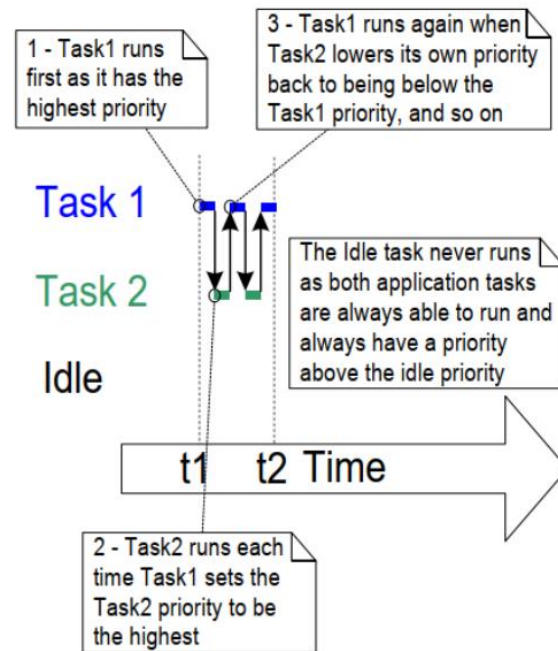
    vTaskStartScheduler();
    for( ;; ) { }
}
```

- 자기 자신의 Priority를 조회하거나 변경할 때에는 NULL Handle을 사용할 수 있다.
- 다른 Task를 제어할 때에는 생성 시 얻은 Task Handle이 필요하다.



Task 관리

- 두 Task의 상대 Priority를 실행 중에 바꾸며 Scheduler의 동작을 확인한다.
- Task 1이 Task 2의 Priority를 자기보다 높게 올린다.



task execution when running Example 4.8

Figure 4.14 The sequence of

- Task 2가 즉시 Running 상태가 되어 Task 1을 Preempt한다.
- Task 2가 자신의 Priority를 낮추면 Task 1이 즉시 Running 상태로 돌아온다.



Task 관리

□ vTaskDelete()는 Task를 삭제한다.

```
void vTaskDelete( TaskHandle_t xTaskToDelete );
```

- FreeRTOSConfig.h에서 INCLUDE_vTaskDelete를 1로 설정해야 사용할 수 있다.
- 삭제한 Task는 더 이상 존재하지 않으며 다시 Running 상태가 될 수 없다.
 - xTaskToDelete에 삭제할 Task Handle을 전달하며, NULL은 호출 Task 자신을 뜻한다.
- 자기 삭제 Task의 TCB와 Stack 등 Kernel Memory는 Idle Task가 해제한다.
- Task 코드에서 별도로 할당한 Memory와 Resource는 개발자가 직접 해제해야 한다.
- Task를 계속 생성하고 삭제하는 설계보다 기존 Task를 재사용하는 방법을 우선 고려한다.



Task 관리

- Task 2의 생성·자기 삭제와 Idle Task의 Memory 정리 과정을 반복한다.
 - main()이 Priority 1의 Task 1을 생성한다.
 - Task 1이 Priority 2의 Task 2를 생성하면 Task 2가 즉시 실행한다.
 - Task 2는 자기 Handle을 사용하여 스스로 삭제한다.
 - Task 1은 다시 실행되어 100 ms 동안 vTaskDelay()로 Blocked 상태에 들어간다.
 - 그동안 Idle Task가 삭제된 Task 2의 Kernel Memory를 정리한다.
 - Task 1이 Ready 상태가 되면 Idle Task를 Preempt하고 Task 2를 다시 생성한다.



Task 관리

- main()은 Task 1만 생성하고 Scheduler를 시작한다.

```
TaskHandle_t xTask2Handle = NULL;
```

```
int main( void )
```

```
{
```

```
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );
```

```
    vTaskStartScheduler();
```

```
    for( ;; ) { }
```

```
}
```

- Task 1이 Priority 2의 Task 2를 생성한다.
 - 더 높은 Priority의 Task 2가 즉시 실행한다.
 - Task 2는 자기 Handle을 사용하여 스스로 삭제한다.



Task 관리

□ Task 1은 Task 2를 생성한 뒤 지연하고, Task 2는 자기 자신을 삭제한다.

```
void vTask1( void * pvParameters )
{
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100UL );
    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        xTaskCreate( vTask2, "Task 2", 1000, NULL, 2,
&xTask2Handle );
        vTaskDelay( xDelay100ms );
    }
}
```

```
void vTask2( void * pvParameters )
{
    vPrintLine(
        "Task 2 is running and about
to delete itself" );
    vTaskDelete( xTask2Handle );
}
```

□ 동적 메모리로 생성한 Task가 자기 자신을 삭제하면

- Idle Task가 TCB와 Stack 등 Kernel이 할당한 Memory를 해제한다.
- Task 코드에서 별도로 할당한 Memory나 Resource는 개발자가 직접 해제한다.



Task 관리

- Task 1이 Task 2를 만들고, Task 2가 자기 자신을 삭제한다.

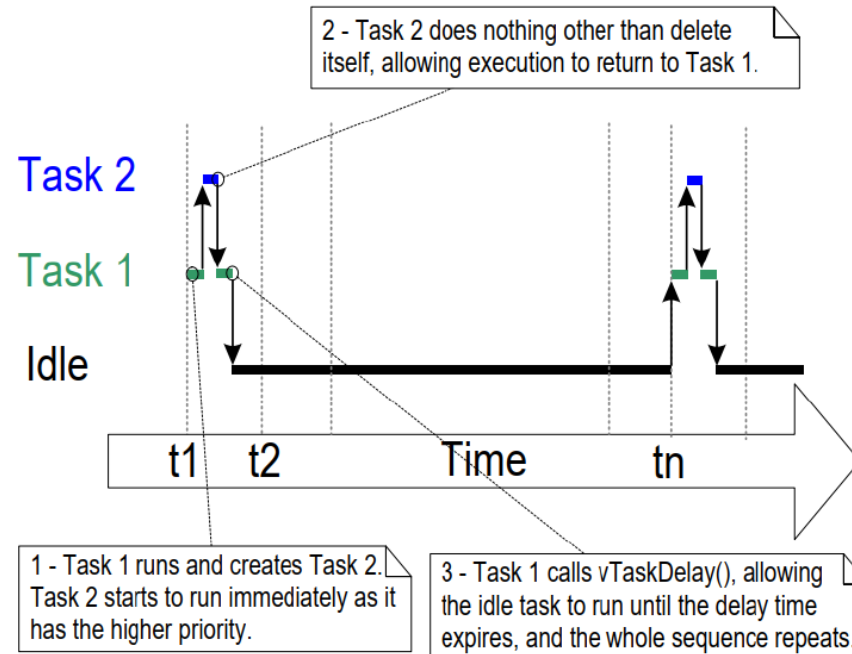


Figure 4.17 The execution sequence for Example 4.9

- Task 1이 `vTaskDelay()`로 Blocked 상태에 들어간다.
- 그동안 Idle Task가 삭제된 Task 2의 Kernel Memory를 정리한다.



Task 관리

□ Thread Local Storage(TLS)는 각 Task의 TCB 안에 Application이 임의의 데이터를 저장할 수 있게 한다.

○ Reentrant하지 않은 함수가 전역 변수에 저장하던 상태를 Task별로 분리할 때 자주 사용한다.

설정 또는 API	역할
configUSE_NEWLIB_REENTRANT	newlib의 Task별 Reentrancy 지원
configUSE_PICOLIBC_TLS	picolibc의 Task별 TLS 지원
configUSE_C_RUNTIME_TLS_SUPPORT	사용자 정의 C Runtime TLS 활성화
configNUM_THREAD_LOCAL_STORAGE_POINTERS	Application TLS Pointer 배열 크기
pvTaskGet... / vTaskSet...	Application TLS Pointer 조회와 설정

□ 대표적인 예는 errno이다.

○ strtod(), strtol() 같은 함수는 상세 오류 상태를 errno에 기록한다.

○ TLS는 과도한 Critical Section 의존을 줄이는 데 적합하다.



Task 관리

☐ Reentrant Function

- 여러 Thread 또는 Task에서 동시에 호출해도 부작용 없이 안전하게 동작한다.
- Non-Reentrant 함수를 TLS 없이 사용하면 공유 상태 보호를 위해 Critical Section이 필요할 수 있다.

라이브러리	FreeRTOSConfig.h 설정
newlib	configUSE_NEWLIB_REENTRANT
picolibc	configUSE_PICOLIBC_TLS

☐ 대표적인 예는 ISO C 표준 라이브러리와 POSIX 시스템이 사용하는 errno이다.

- strtod(), strtol() 같은 함수는 상세 오류 상태를 errno에 기록한다.
- 멀티스레드 환경에서는 Task 또는 Thread별 errno가 필요하다.



Task 관리

□ Application이 자체 C Runtime TLS를 구현하려면 FreeRTOSConfig.h에 다음 매크로를 정의한다.

매크로	역할
configUSE_C_RUNTIME_TLS_SUPPORT	C Runtime TLS 지원 활성화
configTLS_BLOCK_TYPE	TLS Data를 저장할 C 자료형 정의
configINIT_TLS_BLOCK	TLS Block 초기화
configSET_TLS_BLOCK	Task Switch-In 시 TLS Block 설정
configDEINIT_TLS_BLOCK	TLS Block 해제 또는 종료 처리



Task 관리

□ C Runtime TLS와 별도로 Application 전용 Pointer 배열을 각 TCB에 포함할 수 있다.

○ configNUM_THREAD_LOCAL_STORAGE_POINTERS를 0보다 큰 값으로 설정하면 활성화한다.

```
void * pvTaskGetThreadLocalStoragePointer(  
    TaskHandle_t xTaskToQuery,  
    BaseType_t xIndex  
);  
  
void vTaskSetThreadLocalStoragePointer(  
    TaskHandle_t xTaskToSet,  
    BaseType_t xIndex,  
    void * pvValue  
);
```



Task 관리

☐ Running

- 실제로 CPU 시간을 사용하고 있는 Task이다.
- 단일 Core에서는 한 시점에 하나의 Task만 Running 상태가 될 수 있다.

☐ Ready

- Blocked 또는 Suspended 상태가 아니며 실행 준비가 된 Task이다.
- Scheduler는 Ready Task 중 가장 높은 Priority의 Task를 Running 상태로 선택한다.

☐ Blocked 상태와 Event

- Blocked Task는 Event를 기다리며, Event가 발생하면 자동으로 Ready 상태로 이동한다.
- Temporal Event는 자연 시간 만료와 같은 시간 기반 사건이다.
- Synchronization Event는 Queue, Semaphore, Task Notification, Event Group 등을 통해 전달된다.
- FreeRTOS는 AMP와 SMP도 지원하지만 이 절은 단일 Core를 기준으로 한다.



Task 관리

☐ Scheduling Algorithm

- Ready 상태 Task 중 어느 Task를 Running 상태로 전환할지 결정하는 Software 규칙이다.

Scheduling Algorithm	Prioritized	configUSE_PREEMPTION	configUSE_TIME_SLICING
Preemptive + Time Slicing	Yes	1	1
Preemptive without Time Slicing	Yes	1	0
Cooperative	No	0	Any

☐ 모든 단일 Core 설정에서 같은 Priority를 가진 Ready Task는 차례로 실행 기회를 얻는다.

- 이를 흔히 Round Robin Scheduling이라고 한다.
- Round Robin은 정확히 같은 CPU 시간을 보장하지 않고 실행 순서를 번갈아 준다는 의미이다.
- configUSE_TICKLESS_IDLE도 Scheduling에 영향을 주지만 이 절에서는 0으로 가정한다.



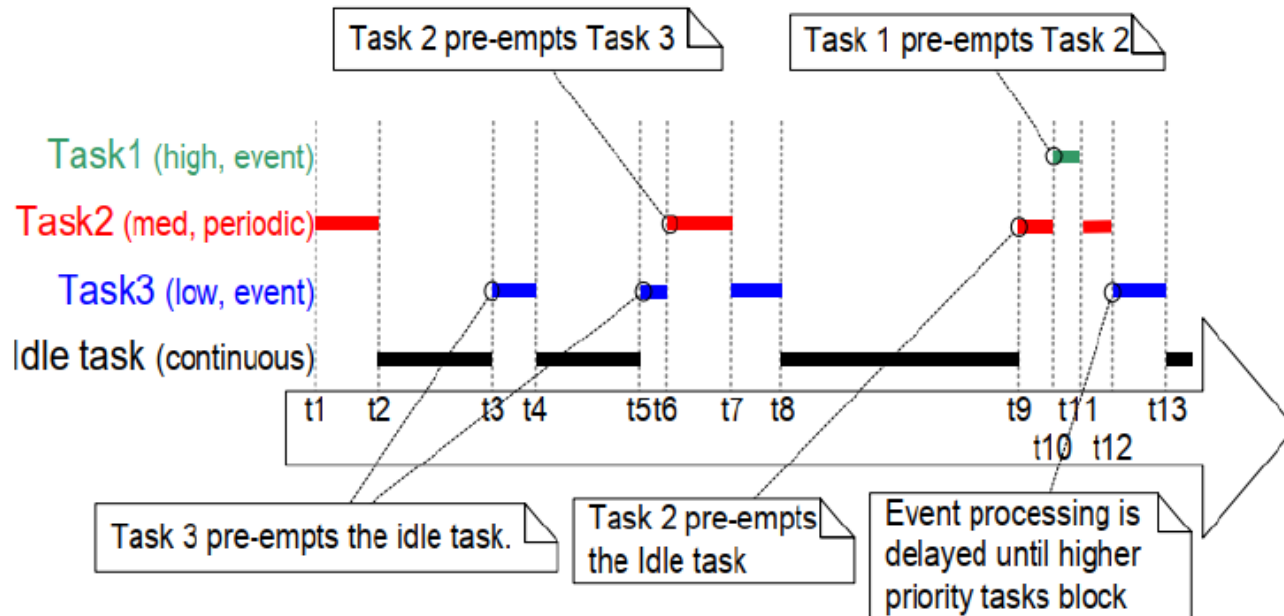
Task 관리

- ☐ configUSE_PREEMPTION=1, configUSE_TIME_SLICING=1
 - Fixed Priority
 - Scheduler가 Task에 할당된 Priority를 자동으로 바꾸지 않는다.
- ☐ Task 자체는 API로 자신이나 다른 Task의 Priority를 변경할 수 있다.
 - Preemptive
 - 더 높은 Priority Task가 Ready 상태가 되면 현재 Task를 즉시 Preempt하며, Tick 시점이 아니어도 발생할 수 있다.
- ☐ Time Slicing
 - 같은 Priority의 Ready Task가 여러 개이면 Tick마다 새 Task를 선택한다.
 - 한 Time Slice는 두 Tick Interrupt 사이의 시간과 같다.



Task 관리

□ Event 발생으로 Ready 상태가 된 높은 Priority Task가 현재 Running Task를 즉시 Preempt한다.



- Task 1 : 시스템에서 가장 높은 Priority의 Event-Driven Task
- Task 2 : Task 3보다 높고 Task 1보다 낮은 Priority의 Periodic Task
- Task 3은 낮은 Priority의 Event-Driven Task이고, Idle Task는 최저 Priority이다.



Task 관리

- Idle Task와 Task 2는 Priority 0의 Continuous Processing Task로 Tick마다 CPU 시간을 번갈아 사용한다.
- 높은 Priority의 Task 1은 Event가 발생하면 즉시 Preempt한다.

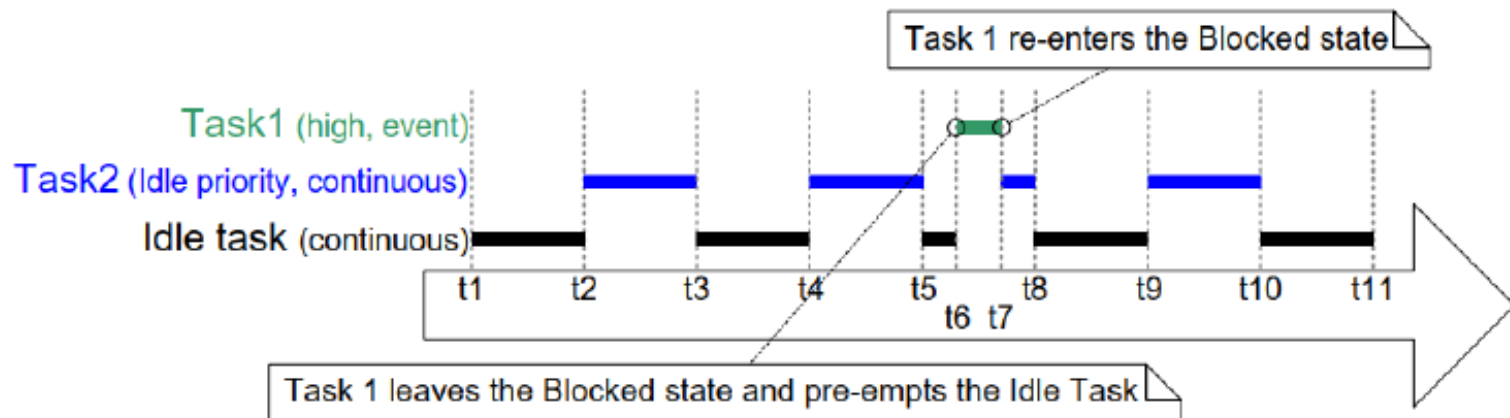


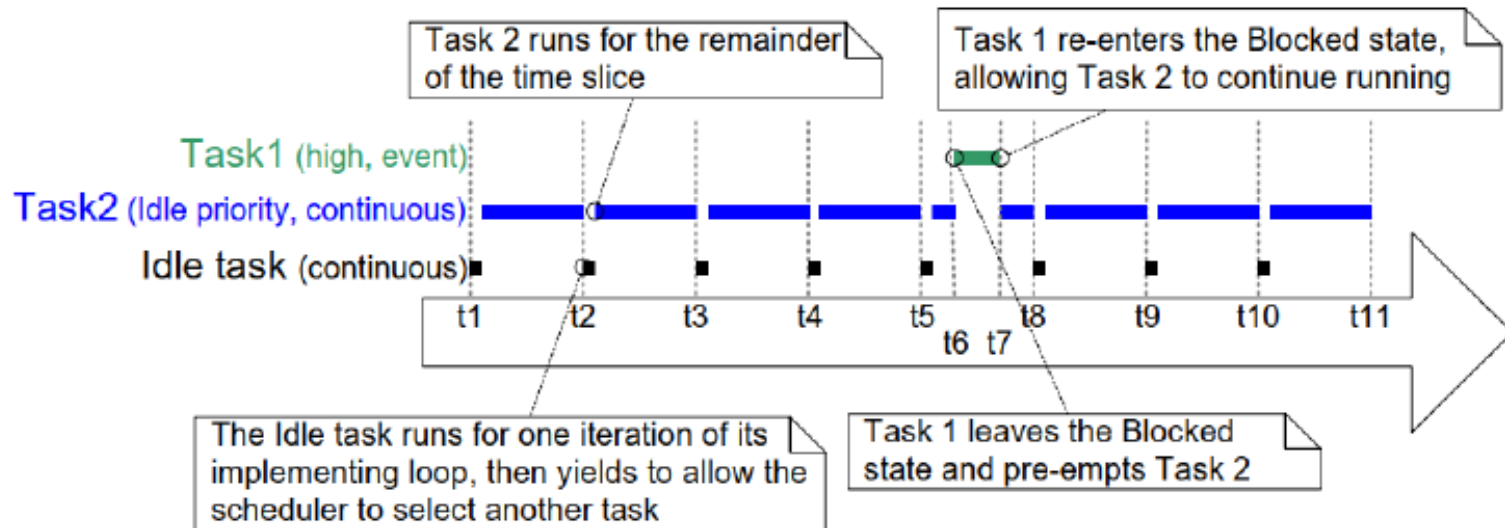
Figure 4.19 Execution pattern highlighting task prioritization and time slicing in a hypothetical application in which two tasks run at the same priority

- configIDLE_SHOULD_YIELD=0이면 Idle Task가 자기 Time Slice 전체를 사용한다.



Task 관리

- configIDLE_SHOULD_YIELD=1이면 같은 Priority 0의 Ready Task가 있을 때 Idle Task가 남은 Time Slice를 양보한다.



- 그 다음 Task는 남은 Time Slice만 실행할 수 있다.



Task 관리

□ configUSE_PREEMPTION=1, configUSE_TIME_SLICING=0

○ 같은 Priority Task 사이에서 Tick마다 강제로 교체하지 않는다.

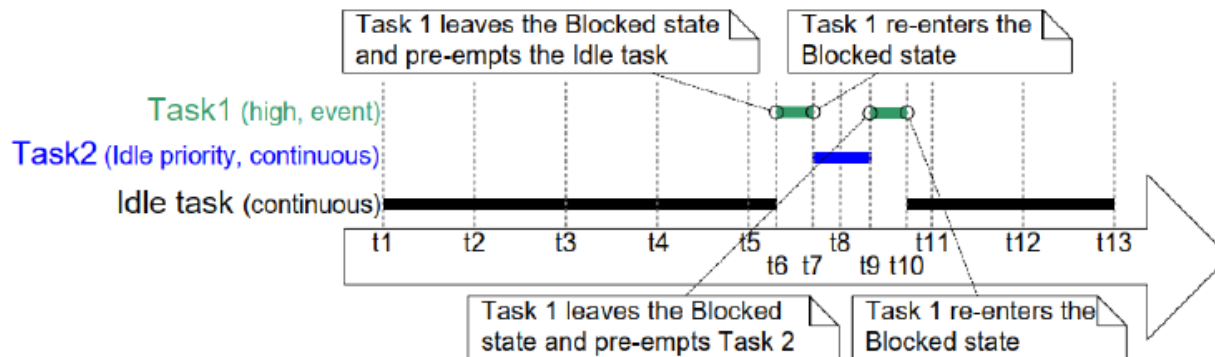


Figure 4.21 Execution pattern that demonstrates how tasks of equal priority can receive hugely different amounts of processing time when time slicing is not used

- Scheduler가 새 Task를 선택하는 경우
 - 더 높은 Priority의 Task가 Ready 상태가 된다.
 - 현재 Running Task가 Blocked 또는 Suspended 상태로 들어간다.
- Context Switch 횟수가 줄어 Scheduler Overhead가 감소한다.
- 같은 Priority Task가 서로 다른 CPU 시간을 받을 수 있어 신중하게 사용해야 한다.



Task 관리

❑ configUSE_PREEMPTION=0

○ Cooperative Scheduler에서는 Task가 자동으로 Preempt되지 않는다.

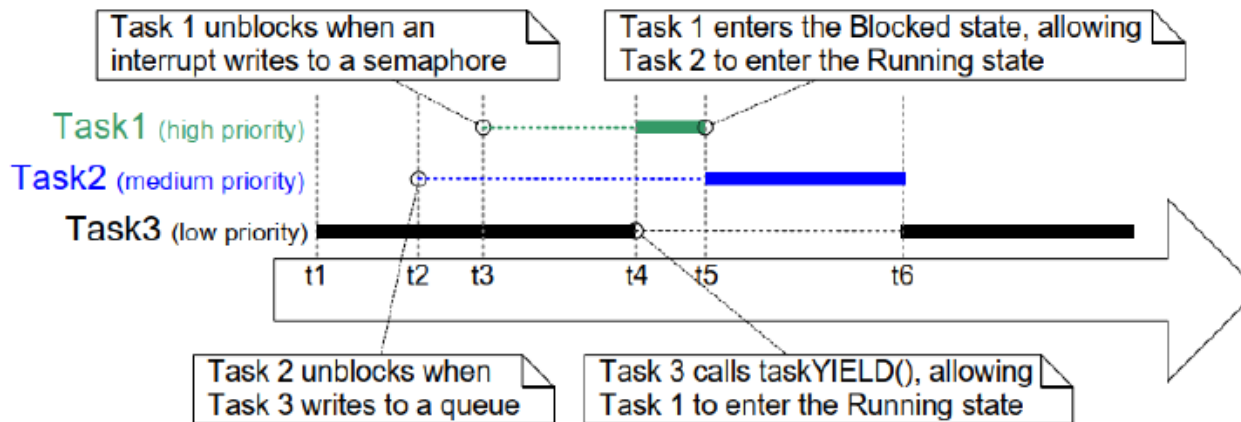


Figure 4.22 Execution pattern demonstrating the behavior of the cooperative scheduler

○ Application ISR이 명시적으로 Context Switch를 요청하지 않는다고 가정한다.

➤ Running Task가 Blocked 상태로 들어갈 때 Context Switch가 발생한다.

➤ Running Task가 taskYIELD()를 호출할 때 Context Switch가 발생한다.

○ Task가 자동으로 Preempt되지 않으므로 Time Slicing을 사용할 수 없다.

○ 높은 Priority Task도 현재 Task가 Block 또는 Yield할 때까지 기다려야 한다.



Task 관리

- ☐ Task 1이 UART에 "abcdefghijklmnop"를 쓰기 시작해 "abcdefg"까지 출력한 뒤 Running 상태를 잃는다.
 - Task 2가 Running 상태가 되어 "123456789"를 모두 출력한다.
 - Task 1이 다시 Running 상태가 되어 나머지 "hijklmnop"를 출력한다.

Task 1 : abcdefg hijklmnop
Task 2 : 123456789

UART : abcdefg123456789hijklmnop

- ☐ Preemptive Scheduler
 - Running Task가 Resource를 일관되지 않은 상태로 둔 순간에도 Preempt될 수 있다.
- ☐ Cooperative Scheduler
 - Resource가 불완전한 상태일 때 Yield하지 않도록 설계하기 쉽다.
 - 다만 현재 Task가 Block 또는 taskYIELD()를 호출할 때까지 높은 Priority Task의 실행이 늦어진다.



Task 관리

- ☐ Task는 Running / Ready / Blocked / Suspended 상태를 전환한다.
- ☐ Scheduler는 실행 가능한 가장 높은 Priority Task를 선택한다.
- ☐ vTaskDelay()는 상대 지연, vTaskDelayUntil()은 고정 주기 실행에 적합하다.
- ☐ Idle Task는 Priority 0으로 자동 생성되며 여유 CPU 사용과 삭제 Task
- ☐ Resource 정리를 담당한다.
- ☐ FreeRTOS는 Time Slicing을 사용하는 Preemptive, 사용하지 않는 Preemptive, Cooperative Scheduling을 지원한다.

Where Possibility Begins

가능성이 미래가 되는 곳

HINT는 현대자동차그룹의
미래 모빌리티 산업 현장과 기술 경험을 통해
미래 인재로의 성장을 지원하는 프로그램입니다.

미래 사회에 필요한 기본 역량은 물론,
모빌리티 핵심 분야인 임베디드 AI, 제조 지능화 및
기업문화 경험을 통해 미래 모빌리티 산업을 이끌어갈
청년 인재의 성장을 지원합니다.

